

TCP/IP Client and Server

Overview

Rhapsody supports receiving and sending messages over a networked environment utilizing the TCP/IP communication protocols and provides communication points able to act in a client or in a server role.

- **TCP Client:** supports environments where the Rhapsody engine initiates the network connection prior to sending or receiving messages.
- **TCP Server:** supports environments where the Rhapsody engine establishes a listener and waits for connections from external hosts.

Supported Modes

The TCP communication points support all communication modes; that is, Input, Output, Bidirectional, In->Out, & Out->In. The TCP Server will normally be configured in either Bidirectional or In->Out modes. The TCP Client will normally be configured in either Out->In or Bidirectional modes. Many external servers are only able to safely process one message at a time, when connecting to such a system with a TCP Client communication point, you will need to use Out->In mode.

Message traffic over TCP communications requires that the message be transmitted as TCP packets. Depending on the network configuration and the size of the message, this typically requires that the message is broken into smaller parts, transmitted and then re-assembled. This is generally transparent to the sending and receiving applications, although there are some scenarios where small message sizes may cause issues (notably when the message size is smaller than the network packet).

JavaScript TCP Communication Points

There are JavaScript versions of both the TCP Client and TCP Server communication point. These are configured using JavaScript and are designed to implement TCP-based connections where an existing communication point does not exist.

The TCP Connection

Messaging over a TCP connection requires the following:

- Establishment of a TCP connection between the two hosts (network level)

- Management of the message stream between the two hosts (application level)

Establishing the TCP connection between the hosts requires the definition of the host and port to connect to. Normally, the host initiating the connection will utilize a randomly allocated port (with a port number in the range of 1025 to 65535) as the local end of the connection (ports below 1025 are reserved for system use).

In the case of a client connection (that is, where the Rhapsody host initiates the connection), the configuration of the communication point requires that the remote host and port number on the remote host be specified. The remote port number should be fixed and allocated to this connection only.

In the case of a server (listener) connection, the port on which the Rhapsody engine will listen for connections must be specified.

A TCP connection is made by connecting to the port on the remote host at the network socket level (that is, Layer 3 of the OSI network model – see [Wikipedia\(opens in a new tab\)](#) for further information). This requires the network socket on the remote host to be configured to accept connections and that there be no firewall blocking that host and port.

Establishing the connection involves the exchange of packets at the network level to establish the willingness of both parties to communicate, and incurs an overhead of a number of network packets to establish the connection. The overhead is usually not significant in comparison to the number of packets required for message transmission. In some specific situations, it may be significant. For example, where the connection needs to be made for each message and the message load is moderate to high, the overhead of connection creation will be high.

The Input Connection

When configured for input, a TCP communication point essentially opens a network port and waits for a remote server to connect to the port (the details of how this happens are different for the client and server communication points).

In Input mode, the TCP communication point is able to accept multiple connections from remote hosts (defined on the General tab of the Properties dialog), and is also able to manage a connection backlog. The backlog is utilized when all the available connections are in use and a remote host connects wanting to send messages. In this scenario, the communication point allows the connection to occur at the network level and queues it to accept messages if one of the active messaging connections is closed.

In Bidirectional mode, the communication point also invokes implicit connection tracking, allowing it to send the output for a message back on the same connection as the input

message was received. This is significant when there are multiple hosts connected to a single communication point, as it ensures that the appropriate response is sent back to each upstream host. Other options send the response on any connection and manage the behavior if the connection has closed.

The Output Connection

In Output mode, the communication point is generally the initiator of the connection, opening the connection to the remote host and port. This means that it is effectively the controller for the connection, with the retry behavior providing the means to manage the connection if it closes unexpectedly (that is, for any reason other than the communication point itself closing the connection).

The TCP communication point in Output mode is typically configured to maintain a constant connection. This assumes that there is a regular flow of messages from the communication point at least every few seconds. In this case, the overhead of closing and re-opening the connection for each message would be significant in terms of time and network traffic.

Where the message load is lower, the communication point may be configured to create the connection, send the message and then close the connection for the transmission of each message.

TCP Connection Management

Connection Types

Rhapsody distinguishes three types of connections:

- TCP connections: these occur at the network level
- Communication point: these occur at the application level and are displayed in the Rhapsody Management Console. They do not necessarily equate to network level communications, as the connection is registered when the communication point is placed into the running state. The communication point will show the connection while the TCP connection is starting or is in the re-try state
- Messaging: when the communication point is in Out->In or In->Out mode, the message flow may be monitored and re-tries executed if a response isn't received within the defined period

TCP Connection Management

Rhapsody distinguishes three types of connections:

- In Input mode, the engine monitors the state of the connection. If a connection closes or is dropped (for example, by a network issue), the engine will manage the closure as gracefully as possible and then reset the network socket so that it is able to receive a new connection.

In Output mode, the engine has more control over the connection. In particular, the configuration supports the specification of the retry (that is, re-connection attempt) behavior, providing three modes of re-connection and control of the frequency and number of attempts. The re-connection behavior supports:

- Immediate retry: attempt re-connection immediately when a failure occurs; this assumes that the root cause of the connection failure is relatively transient and that the network connection to the remote host is highly fault-tolerant and of high availability
- Linear retry: pause between retry attempts as defined by the retry delay setting; this mode assumes that network connection issues resolve after a finite time of a few seconds or more
- Exponential retry: pause for an increasing period of time after each failure; this assumes that the likelihood of a rapid resolution for the connection issue decreases with time, and that it is therefore preferable to delay the retry attempts further

The communication point configuration also permits specification of the number of retries to attempt before stopping the communication point. The number of retries may be set to Infinite if it is desired to ensure that message traffic restarts as soon as possible after the network communications are restored.

While the communication is not connected to the remote host and is engaged in a retry cycle, it is placed into the Retrying status; this status will be visible in the Management Console.

Connection Tracking

Communication points support multiple connections, which, in the case of Input mode connections, may be initiated from multiple remote hosts. When the communication point is configured in bidirectional or synchronous modes, it is generally important that the return message is sent to the host from which the message originated.

Rhapsody provides implicit connection tracking in these cases so that the response will be returned to the correct host. Additionally, the configuration provides support for managing the message if the connection has been closed or has failed before the response can be sent.

Specifying a Hostname

The **hostname** required as part of the connection details may be specified as either a hostname or as an IP address. Where a hostname is utilized, Rhapsody resolves that to its IP address via DNS.

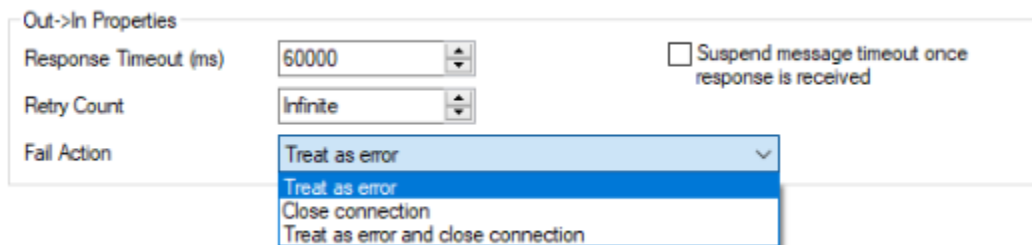
Rhapsody does not cache the results of a DNS lookup. This ensures that where DNS failover is used, the change in the IP address becomes available to Rhapsody quickly. In some environments, caching can occur in other systems that will result in a delay between the change occurring and the change becoming visible to Rhapsody.

Utilizing a Rhapsody variable for the hostname will also simplify updating the configuration if the IP address of the host does change.

Messaging with Synchronous Modes

Rhapsody provides additional support for managing communications at the message level by means of the Out->In properties of the **General** configuration tab. The synchronous messaging modes block receipt or sending of messages until the current operation completes, and these parameters ensure that processing is able to continue if a message response issue occurs.

In Out->In mode, the communication point sends a message and then waits for a response before the next message can be sent. The Out->In properties define a timeout for the response message, and define the resend behavior and the behavior if the number of resends is exceeded.



Out->In Properties

Response Timeout (ms) 60000 ☐ Suspend message timeout once response is received

Retry Count Infinite

Fail Action

- Treat as error
- Treat as error
- Close connection
- Treat as error and close connection

In In->Out mode (that is, the receipt of messages is blocked until the message is sent out by the communication point), only the **Response Timeout** parameter is enabled. If the message cannot be transmitted by the communication point within the timeout period, the message is sent to the error queue or optionally may be discarded.

Message Tracking functionality may override synchronous mode messaging behavior, and provides a finer grained ability to manage the message flow.

Messaging and the wrapper

Data is received over a TCP connection as a continuous stream with no information as to the organization of the content. The TCP communication points provide configuration options that let us insert characters into the data stream to mark the start and end of each message.

These characters define the start and end of a specific message in the data stream and are referred to as the wrapper. They generally do not form part of the message and are normally stripped from the data stream before the message is processed by Rhapsody. The wrapper characters may be specified as:

- **Minimal:** the HL7 [Minimal Lower Layer Protocol](#) (MLLP); the character 0x0B (hexadecimal 0B) precedes the message, and the characters 0x1C 0x0D (hexadecimal 1C, 0D) terminates the message
- **User:** User-defined header and trailer characters
- **Hybrid:** MLLP with the addition of a checksum
- **None**

User Defined

When the wrapper is set as User, the following parameters are exposed and must be defined:

- **Header:** the unique character sequence which will define the start of the message
- **Trailer:** the unique character sequence which will define the end of the message
- **Endianness:** specifies the ordering of bytes in the data stream; that is, whether the most significant byte is presented first (Big) or the least significant byte is presented first (Little)

A Secure TCP Connection

Connection Security and Certificate Management is covered in detail in the *Rhapsody Services* module.

Establishing a secure TCP connection requires a little more overhead than a standard connection as the hosts must reach agreement on the encryption method and specific encryption key to be used for this session. TLS (Transport Layer Security) is preferred. When establishing the connection, the process is:

1. Client sends SSL details, cipher, and session data.
2. 2

Server responds with its requirements, including its certificate.

3. 3

Client authenticates server from the details supplied.

4. 4

Client generates session secret, encrypts with the server's public key and transmits to the server.

5. 5

Client and server use the secret to generate the session key to be used for this session.

6. 6

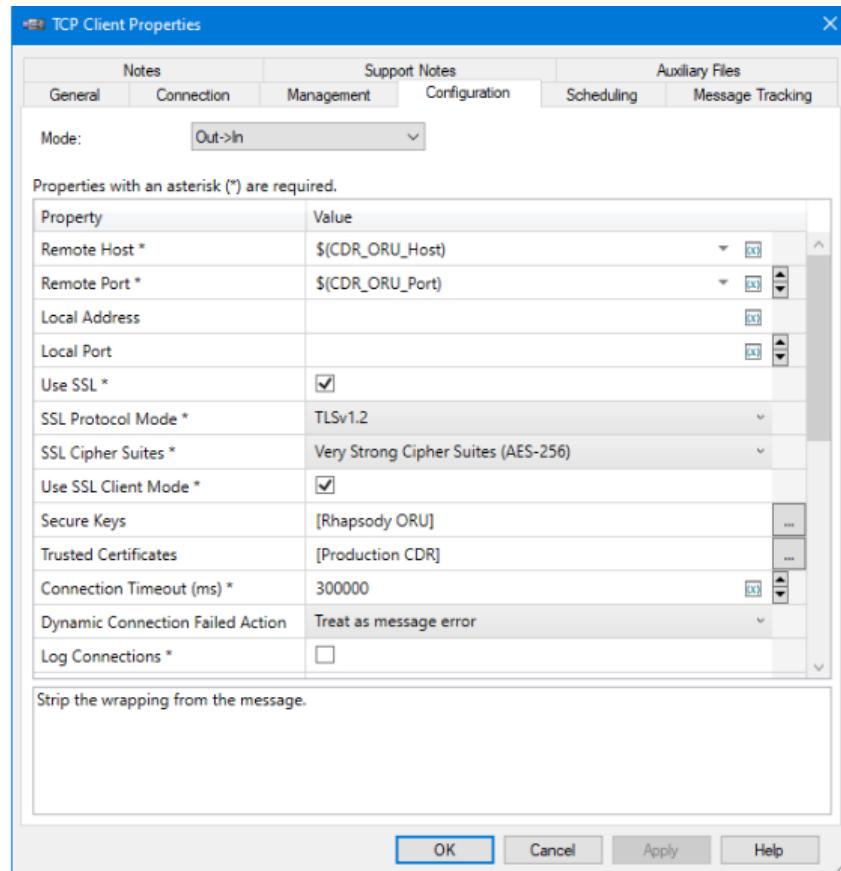
Client and server handshake agreement to use the session keys.

7. 7

All communications from this point are encrypted with the session keys.

Configuring for Secure Communications

The TCP communication points support utilizing secure communications by means of the SSL parameters. These are enabled by setting the **Use SSL** parameter to true.



By default, the SSL Protocol will be set to TLS and SSL Client mode will be set to false (client mode defines which of the hosts will act as the client to initiate the connection). The Keys to be used while establishing the connection may then be selected from the Certificate Manager:

- **Secure Keys:** the Rhapsody server private keys used to decrypt its communications from the remote host (encrypted by the server's public keys on the remote host).

If a TCP Server communication point is configured without secure keys, it will attempt to operate the connection in anonymous server authentication mode (provided the selected SSL Cipher Suites property permits this). This should be avoided as it allows [man-in-the-middle](#) attacks to be performed.

- **Trusted Certificates:** the public keys used to encrypt the session establishment communications to the remote host.

Troubleshooting

The Rhapsody engine logs all connection failures to the system text and binary logs. The text log is named log.txt and located in the log directory of the Rhapsody application path, and the binary log is displayed on the component pages and in the System Log page of the Rhapsody Management Console. By default, the engine logs component errors at the INFO level which are always reported. For more detail on a specific issue, temporarily raise the log level to DEBUG or TRACE on the component details page.

Note that TRACE level logging will generate large amounts of log detail; the log level should only be set to TRACE for short periods to resolve a specific problem.

The Rhapsody log presents the Java stack trace for each error. These can be complex and daunting, but the key to understanding is to isolate the **Caused by** lines, as these indicate the root of the issue. A stack trace may have multiple **Caused by** lines, so analyzing an error requires extracting all the lines and identifying the first in the sequence. This usually indicates the primary cause of failure.

Common TCP Component Error Messages

The most common issues presented for TCP connections are

- Connection failed: either a connection to a remote host could not be achieved or an inbound connection failed
- Exceeded connection retry limit, shutting down communication point; occurs when the retry limit is exceed – the engine automatically shuts the communication point down
- IOException: Connection reset

Connection Reset Stack Trace

```
2012-08-28 11:31:49,112 INFO [werscribeGOut [108, 6805] OUT_IN] [
CommunicationPoint.108] IOException occurred: Connection reset
```

```
com.orchestral.rhapsody.module.communicationpoint.ConnectionFailedException:
IOException occurred: Connection reset
```

```
at
```

```
com.orchestral.rhapsody.resource.tcp.TCPClientTransport$TCPClientTransportConnectio
n.read(TCPClientTransport.java:20)
```

at
com.orchestral.rhapsody.resource.tcp.TCPClientTransport\$TCPClientTransportConnection.read(TCPClientTransport.java:130)

at
com.orchestral.rhapsody.resource.tcp WrapperProtocolFactory\$WrapperProtocol.receiveMessage(WrapperProtocolFactory.java:15)

at
com.orchestral.rhapsody.module.communicationpoint.ProtocolTransportCommunicationPoint\$ProtocolTransportCommunicationPointConnection.receiveMessage(ProtocolTransportCommunicationPoint.java:17)

at
com.orchestral.rhapsody.execution.communicationpoint.spi.ConnectionInternal.receiveMessage(ConnectionInternal.java:101)

at
com.orchestral.rhapsody.execution.communicationpoint.spi.ConnectionInternal.receiveMessageInternal(ConnectionInternal.java:48)

at
com.orchestral.rhapsody.execution.communicationpoint.spi.connection.OutputInputConnectionExecutor.receiveMessage(OutputInputConnectionExecutor.java:123)

at
com.orchestral.rhapsody.execution.communicationpoint.spi.connection.OutputInputConnectionExecutor.execute(OutputInputConnectionExecutor.java:47)

at
com.orchestral.rhapsody.execution.communicationpoint.spi.SingleConnectionExecutor.run(SingleConnectionExecutor.java:18)

at java.util.concurrent.ThreadPoolExecutor\$Worker.runTask(ThreadPoolExecutor.java:885)

at java.util.concurrent.ThreadPoolExecutor\$Worker.run(ThreadPoolExecutor.java:907)

at
com.orchestral.rhapsody.execution.communicationpoint.spi.ConnectionThreadPool\$ConnectionThread.execute(ConnectionThreadPool.java:4)

at
com.orchestral.rhapsody.execution.communicationpoint.spi.AbstractCommunicationPointThread.run(AbstractCommunicationPointThread.java:17)

Key elements:

IOException occurred: Connection reset: This indicates the TCP network connection was reset by the remote host, causing the TCP session to fail. The communication framework will attempt to create a new connection to the remote host.

This is normally a network issue and is likely to be transient. If it recurs frequently, it will require discussion with the network administrators.

2012-08-28 16:21:18,723 INFO [werscribeGOut [108] - Controller] [CommunicationPoint.108] Communication Point Connection Failed

com.orchestral.rhapsody.module.communicationpoint.ConnectionFailedException: IO error reconnecting.

at

com.orchestral.rhapsody.resource.tcp.TCPClientTransport\$TCPClientTransportConnection.connectToSocket(TCPClientTransport.java:129)

at

com.orchestral.rhapsody.resource.tcp.TCPClientTransport\$TCPClientTransportConnection.reconnect(TCPClientTransport.java:163)

at

com.orchestral.rhapsody.resource.tcp.TCPClientTransport\$TCPClientTransportConnection.<init>(TCPClientTransport.java:78)

at

com.orchestral.rhapsody.resource.tcp.TCPClientTransport.getConnection(TCPClientTransport.java:14)

at

com.orchestral.rhapsody.module.communicationpoint.ProtocolTransportCommunicationPoint.getConnection(ProtocolTransportCommunicationPoint.java:132)

at

com.orchestral.rhapsody.modules.standard.communicationpoints.tcp.WrappedTCPClient.getConnection(WrappedTCPClient.java:22)

at

com.orchestral.rhapsody.execution.communicationpoint.spi.CommunicationPointInternal.getConnection(CommunicationPointInternal.java:45)

```
at
com.orchestral.rhapsody.execution.communicationpoint.spi.CommunicationPointInternal
.getConnectionInternal(CommunicationPointInternal.java:35)

at
com.orchestral.rhapsody.execution.communicationpoint.spi.CommunicationPointExecut
or.run(CommunicationPointExecutor.java:186)

at
com.orchestral.rhapsody.execution.communicationpoint.spi.CommunicationPointExecut
or$ControllerThread.execute(CommunicationPointExecutor.java:3)

at
com.orchestral.rhapsody.execution.communicationpoint.spi.AbstractCommunicationPoin
tThread.run(AbstractCommunicationPointThread.java:17)

Caused by: java.net.ConnectException: Connection refused (errno:239)

at java.net.PlainSocketImpl.socketConnect(Native Method)

at java.net.PlainSocketImpl.doConnect(PlainSocketImpl.java:333)

at java.net.PlainSocketImpl.connectToAddress(PlainSocketImpl.java:195)

at java.net.PlainSocketImpl.connect(PlainSocketImpl.java:182)

at java.net.SocksSocketImpl.connect(SocksSocketImpl.java:366)

at java.net.Socket.connect(Socket.java:529)

at
com.orchestral.rhapsody.resource.tcp.TCPClientTransport$TCPClientTransportConnectio
n.connectToSocket(TCPClientTransport.java:68)

... 10 more
```

Key Elements:

Communication Point Connection Failed: Reports the issue.

Caused by: java.net.ConnectException: Connection refused (errno:239): The remote host refused to allow the Rhapsody engine to start a TCP session. This may be because of a firewall at the remote host, or that the port does not have a listening process on it.

Resolution of this error normally requires discussion with the Administrators of the remote host or network.

Shutting Down Communication Point

2012-12-05 14:58:54,965 ERROR [Client 3001 [3196] - Controller] [CommunicationPoint.3196] Exceeded connection retry limit, shutting down communication point

com.orchestral.rhapsody.execution.communicationpoint.spi.RetryLimitExceededException: Communication point exceeded maximum retry count of 5

at

com.orchestral.rhapsody.execution.communicationpoint.spi.LinearConnectionRetryStrategy.connectionFailed(LinearConnectionRetryStrategy.java:12)

at

com.orchestral.rhapsody.execution.communicationpoint.spi.CommunicationPointExecutor.run(CommunicationPointExecutor.java:178)

at

com.orchestral.rhapsody.execution.communicationpoint.spi.CommunicationPointExecutor\$ControllerThread.execute(CommunicationPointExecutor.java:1)

at

com.orchestral.rhapsody.execution.communicationpoint.spi.AbstractCommunicationPointThread.run(AbstractCommunicationPointThread.java:12)

2012-12-05 14:58:54,965 WARN [Client 3001 [3196] - Controller] [CommunicationPoint.3196] Communication point stopped in error: Connection retries exceeded

Key Elements:

Exceeded connection retry limit, shutting down communication point: Indicates that the engine has attempted the connection according to the retry rules and that the number of attempts has exceeded the retry limit. The component is therefore placed into the Stopped in Error state.

Exercise - E-mail Client

Introduction

The email communication point allows Rhapsody to send and receive email messages from external systems.

In Input mode, the **E-mail Client** communication point is utilized to retrieve an email message from a server and populate a message with the email content. For example, it might be used to populate a database with a referral received by e-mail. This information can either form the body of the message or be extracted from an attached file.

In Output mode, the **E-mail Client** communication point is used to pass information originating from the Rhapsody engine into the body of an outgoing e-mail message and/or into a file attached to that message. This feature might be used, for example, to email error reports or to an administrator when specified events, such as a route or communication point shutdown, occur.

Testing

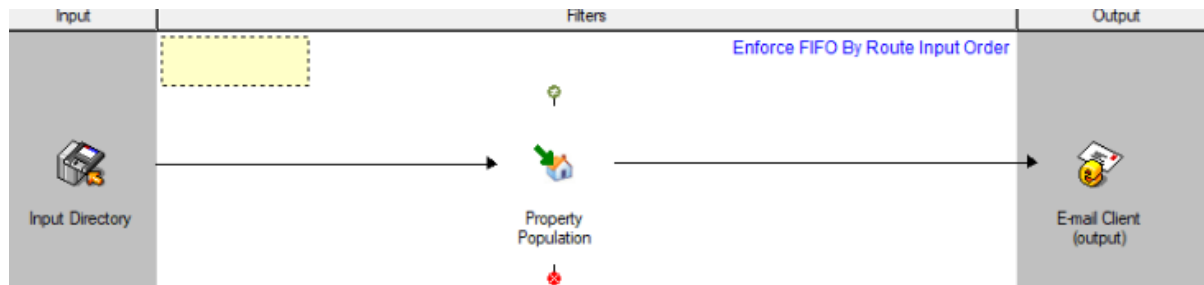
Many organizations run a single email server. This can make testing of the E-mail Client communication point in Rhapsody difficult (especially when Rhapsody is destructively reading messages from an inbox or when large volumes of messages are being generated). A number of different solutions exist for creating small local email test environments that work well with Rhapsody.

The table below presents a selection of email solutions that can be used when testing email in Rhapsody.

System	Description
Papercut SMTP	Simple SMTP test tool. Useful for testing outbound Rhapsody messaging without flooding a user's inbox.
hMailServer	Windows based email server support IMAP, POP3 and SMTP. Requires an SQL server to function. Can be used when Rhapsody needs to read messages from the mail server.
Linux Server running Postfix and Dovecot	Fully featured Unix based solution supporting IMAP, POP3 and SMTP. Can be configured to run in a Virtual Machine. Useful when Rhapsody needs to read messages from the mail server.

Exercise: Email Client in Output Mode

Create an E-mail Client communication point in Output mode and add it to a new route as shown below.



Open the **Property Population** filter's Properties dialog and add a **Literal -> Property Mapping** that identifies the email address of the individual who is to receive these emails. Note that the Property name contains no spaces.

Property	Literal
EmailRecipientAddress	<recipient's email address>

Open the E-mail communication point's **Properties** dialog and complete as shown:

- **Mail Server** - the name of the email server (determine this by consulting with your IT administrators).
- **Protocol** - the protocol used to communicate with the server. By default, this is SMTP (Simple Mail Transfer Protocol) on port 25 although it may be configured as port 587 for your mail server. Determine this by consulting with your IT administrators.
- **From** - the name of the message's sender. This must not contain a space.
- **To** - the email address of the person who is to receive this e-mail. This configuration uses the message property created by the **Property Population** filter to set the recipient's email address. The dollar sign '\$' identifies the **To** value as using a message property set on the route.
- **Subject** - the text that will appear in the message's Subject line.
- **Send As Attachment** - if true, the information will be inserted as an e-mail attachment. If false, the information will be placed into the body of the email.

- **Attachment Name** - If **Send As Attachment** is true, this property sets the attachment's name.
- **Body Text** - If **Send As Attachment** is true, this property defines the body text of the email.

The screenshot shows the 'E-mail Client (output) Properties' dialog box with the 'Configuration' tab selected. The 'Mode' is set to 'Output'. Below the tabs, a note states: 'Properties with an asterisk (*) are required.' The main area contains a table of properties and their values.

Property	Value
Mail Server *	\$(Email_Host)
Protocol *	SMTP
Port	\$(Email_Port)
Use SSL *	☐
SSL Protocol Mode *	
SSL Cipher Suites *	
Trusted Certificates	
Read Timeout (ms) *	10000
Connection Timeout (ms) *	30000
User	
Password	
Sending Mode *	Normal
From *	\$(Email_From)
Reply To	
To *	\$(EmailReceipientAddress)
CC	
Subject	Rhapsody Mail Message
Send Rhapsody Message	☒
Replacement Body Text	
Send As Attachment	☒
Attachment Name *	Rhapsody Message
Body Text	
MIME content type	text/plain
S/MIME *	☐

At the bottom of the dialog are buttons for 'OK', 'Cancel', 'Apply', and 'Help'.

Create a text file containing a readily recognizable block of text then drop a copy into the directory specified by the Input Directory communication point.

What You Should See

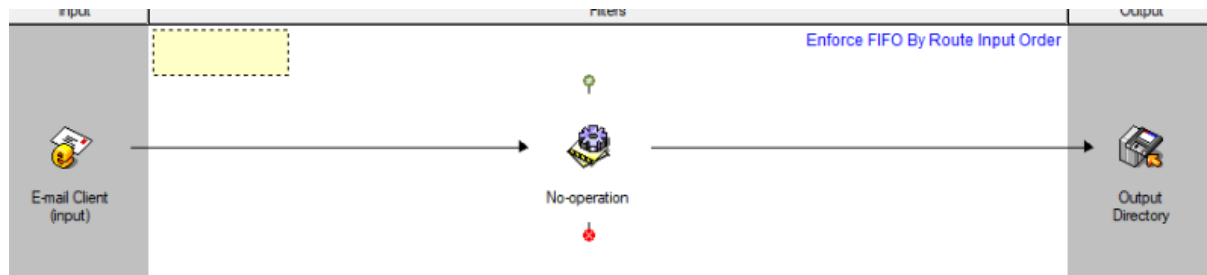
An email will be generated for every file placed in the route's input directory and sent to the recipient defined in the Property Population's configuration.

When **Send As Attachment** is set to true, the content of the file will be available as an attachment to the e-mail. When this property is false, the file's content will appear in the body of the email.

Exercise: E-mail Client in Input Mode

Create an **E-mail Client** communication point in Input mode and add it to a new route as shown below. Add and connect a **Directory** communication point configured to receive messages into an accessible directory.

This exercise will either delete or mark as read messages in the email account used in the exercise. We recommend that this exercise is completed using a test email account.



While the presence of the **No-operation** filter is not critical for the functioning of this route, it is a Rhapsody Best Practice principle that all routes should contain at least one filter to ensure that property extraction occurs correctly.

Open the E-mail communication point's **Properties** dialog and complete as shown (note that you will need the network address, protocol, username and password details for your server):

E-mail Client (input) Properties

Notes | Support Notes | Auxiliary Files

General | Connection | Management | **Configuration** | Scheduling | Message Tracking

Mode:

Properties with an asterisk (*) are required.

Property	Value
Mail Server *	\$(Email_Host)
Store Protocol *	IMAP
POP3 Port	
IMAP Port	143
Use SSL *	☐
SSL Protocol Mode *	
SSL Cipher Suites *	
Trusted Certificates	
Read Timeout (ms) *	10000
Connection Timeout (ms) *	30000
User	\$(Email_Username)
Password	\$(Email_Password)
Folder	Test Messages

OK Cancel Apply Help

- **Mail Server** - the name of the email server. This property is not case-sensitive.
- **Store Protocol** - the protocol used to retrieve mail from the server. By default, this is IMAP on port 143. Alternatively, you can use POP3 on port 110.
- **User** - your email username that provides access to your email application.
- **Password** - Your email password that provides access to your email application.
- **Folder** - when using the IMAP protocol, this sets the folder to read the messages from.
- **Refresh Rate** - the polling delay (in milliseconds) at which the specified folder will be checked for the arrival of new messages.
- **Delete Message** - This determines the behavior Rhapsody uses to read messages from the email server.

When using the POP3 protocol or the IMAP protocol with the option set to true, all

messages will be permanently deleted from your email account.

When using the IMAP protocol with the option set to false, all messages read by Rhapsody will be marked as read on the email server.

- **Message as Attachment** - if true, the content of your e-mail's attachment will be extracted and copied into the body of the message placed in the output directory. If false, the body of the email will form the content of the output message.

Copy a number of emails to the folder you have specified for this exercise; some with information in the body of the email and some with the content in an attachment. Mark the messages as unread, then start the Rhapsody configuration.

What You Should See

The output directory should populate with messages containing the content of incoming emails, regardless of whether that content originated in the body of the email or as an attachment.

Supplementary Exercise

The PID (Patient Identification) in an HL7 message includes a field identifying the patient's email address.

- Create an ADT test message. Ensure the PID.HomePhoneNumber.EmailAddress field contains a valid, accessible email address
- Add the definition file used to create the test message to the **Property Population** filter's **Properties** dialog. Configure its **Field -> Property Mapping** property to extract this value
- Configure the **E-mail Client** communication point's **To** property to use the email address extracted from the message
- Place a copy of the test message in the route's input directory

What You Should See

An email message will be generated and sent to the email address defined in the original message.

Amazon Web Services

Amazon S3

The **Amazon S3** communication point operates in output mode and connects to an Amazon Simple Storage Service (S3) data bucket. This allows messages to be sent for storage in Amazon S3.

Amazon S3 Properties

Notes Support Notes Auxiliary Files

General Connection Management Configuration Scheduling Message Tracking

Mode: Output

Properties with an asterisk (*) are required.

Property	Value
Endpoint URL *	
S3 Bucket Name *	
AWS Access Key Id	
AWS Secret Key	
Object Key Prefix	
Object Key	
Encryption Mode *	Server Side Encryption (AES-256)
Storage Class *	Standard
Canned ACL *	Private
Content Type	
Content Encoding	
Custom Metadata	
Connection Timeout (ms) *	10000
Read Timeout (ms) *	30000

OK Cancel Apply Help

Amazon S3 Properties

Property	Description
Endpoint URL	the URL for the Amazon S3 data bucket
S3 Bucket Name	the name associated with the data bucket
AWS Security Key	Cell 7the access key provided by Amazon
AWS Secret Key	the private key associated with your Amazon account
Object Key Prefix	An optional prefix that is pre-pended to the specified or generated object key for the message. This can be used to emulate a directory structure by providing a prefix that separates path elements using forward slashes (for example, 'dir1/dir2/'). A trailing forward slash is not added to this value
Object Key	The object key to use when storing the message in S3. If this is blank then a type-4 UUID is generated and used as the object key. A message property may be used here by prefixing the name of the message property with \$. Forward slashes may be included in this value to emulate a directory structure in the S3 bucket
Encryption Mode	Determines what type of encryption is applied to the messages uploaded to S3
Storage Class	Determines which S3 storage class should be used for uploaded messages: Standard Reduced Redundancy or Infrequent Access

Canned ACL	Determines which S3 canned ACL should be used on the uploaded message: None, Private, Public Read, Public Write, Authenticated Read, Log Delivery Write, Bucket Owner Read or Bucket owner Full Control
Content Type	Optionally sets the Content-Type header on the uploaded message (for example, text/plain, application/json, etc.). A message property may be used here by prefixing the name of the message property with \$
Content Encoding	Optionally sets the Content-Encoding header on the uploaded message (for example, gzip, etc.). A message property may be used here by prefixing the name of the message property with \$
Custom Metadata	Allows custom metadata to be set on the uploaded message using either message properties or literal values. The Custom Metadata column is the name of the metadata attribute on S3 (without the x-amz-meta- prefix) and is case insensitive. The Value column is either a literal value, or may be a message property by prefixing the name of the message property with \$
Connection Timeout (ms)	the period after which the communication point will cease attempting to connect with Amazon
Read Timeout (ms)	The read timeout, in milliseconds, to use when waiting for a response from Amazon S3

The S3 bucket must be manually created prior to configuring this communication point, as Rhapsody is not able to do this if a specified bucket does not exist.

Amazon SNS

The **Amazon SNS** communication point enables you to connect to an [Amazon Simple Notification Service \(Amazon SNS\)\(opens in a new tab\)](#), a push notification service that can send individual messages or broadcast messages to large numbers of recipients.

Amazon SNS Properties

Notes

Support Notes

Auxiliary Files

General

Connection

Management

Configuration

Scheduling

Message Tracking

Mode:

Output

Properties with an asterisk (*) are required.

Property	Value
Endpoint URL *	
Destination Type *	SNS Topic ARN
SNS Destination	
AWS Access Key Id	
AWS Secret Key	
Notification Subject	
Message Structure *	String
Connection Timeout (ms) *	10000
Read Timeout (ms) *	30000

OK

Cancel

Apply

Help

Amazon SNS Properties

Property	Description
Endpoint URL	The endpoint URL to use to publish messages to SNS
Destination Type	Determines whether the message is delivered to: • SNS Topic ARN – all subscribers of an SNS topic. • SNS Target ARN – directly to a specific SNS target. • Phone Number (SMS) – a specific phone number as an SMS message (the phone number is provided in E164 format)
SNS Destination	The SNS destination for the message. If Destination Type is set to SNS Topic ARN, then the ARN of the SNS topic must be provided. If Destination Type is set to SNS Target ARN, then the ARN of the SNS target must be provided
AWS Access Key ID	The AWS access key identifier to use to send messages to SNS. This can be left blank if this is running in EC2 with an IAM role that grants the required permissions
AWS Secret Key	The period after which the communication point will cease attempting to connect with Amazon
Notification Subject	The optional subject to set on the published notification. Note that this will only be visible to email subscribers
Message Structure	Determines whether the notification is sent as a string or in JSON format. JSON format should only be used when there is a need to send different information to different types of subscribers
Connection Timeout	The connection timeout, in milliseconds, to use when attempting to connect to Amazon SNS
Read Timeout	The read timeout, in milliseconds, to use when waiting for a response from Amazon SNS

Amazon SQS

The Amazon SQS communication point operates in input or output mode and enables you to connect to an [Amazon Simple Queue Service \(Amazon SQS\)](#)(opens in a new tab), a message queuing service.

Input Mode Properties

The screenshot shows the 'Amazon SQS Properties' dialog box with the 'Configuration' tab selected. The 'Mode' is set to 'Input'. A table lists various properties, some of which are required (marked with an asterisk). The 'Refresh Delay (ms) *' is set to 5000, 'SQS Wait Time (s) *' is 20, 'Receive Body As *' is 'String', 'Connection Timeout (ms) *' is 10000, and 'Read Timeout (ms) *' is 30000. There are also fields for 'Input Queue URL *', 'AWS Access Key Id', and 'AWS Secret Key'. The 'Message Attributes' field has a button to open a sub-dialog. At the bottom are 'OK', 'Cancel', 'Apply', and 'Help' buttons.

Property	Value
Input Queue URL *	
AWS Access Key Id	
AWS Secret Key	
Refresh Delay (ms) *	5000
SQS Wait Time (s) *	20
Receive Body As *	String
On Body Decode Error *	
Message Attributes	
Connection Timeout (ms) *	10000
Read Timeout (ms) *	30000

- **Input Queue URL** - The URL of the AWS SQS queue that this communication point will receive messages from
- **AWS Access Key ID** - The AWS access key identifier to use to receive messages from the queue. This can be left blank if this is running in EC2 with an IAM role that grants the required permissions.
- **AWS Secret Key** - The AWS access secret key to use to receive messages from the queue. This can be left blank if this is running in EC2 with an IAM role that grants the

required permissions.

- **Refresh Delay (ms)** - The delay in milliseconds between each SQS queue polling attempt when the previous attempt found no available messages. If long polling is enabled, this is the delay between when the previous polling attempt completes and when the next attempt starts.
- **SQS Wait Time (s)** - Enables long polling if configured to a non-zero value. If no message is available when long polling is enabled, then SQS will hold onto the polling request for up to the specified number of seconds while waiting for a new message to arrive. The maximum allowed value is 20 seconds.
- **Receive Body as** - Determines how the message body is received.
 - String means to receive the message body as a UTF-8 encoded string;
 - Hex Encoded Binary will decode the received message as hex and save it as a binary value;
 - Base64 Encoded Binary will decode the received message as base64 and save it as a binary value.
- **On Body Decode Error** - Determines the action to take if decoding the incoming message body as either hex or base64 fails.
- **Message Attributes** - Names of SQS message attributes that should be retrieved and stored as message properties in Rhapsody. If present, each of these message attributes is stored as a Rhapsody message property with a name prefixed with "sqs:attrib:". Each message can have up to 10 attributes.
 - Name
 - Value
 - Type: String / Number

- **Connection Timeout (ms)** - The connection timeout in milliseconds to use when attempting to connect to the Amazon SQS queue.
- **Read Timeout (ms)** - The read timeout in milliseconds to use when waiting for a response from an Amazon SQS.

Output Mode Properties

The screenshot shows the 'Amazon SQS Properties' dialog box with the 'Configuration' tab selected. The 'Mode' is set to 'Output'. A table lists properties with asterisks indicating required fields. The 'Connection Timeout (ms)' is set to 10000 and 'Read Timeout (ms)' is set to 30000. The 'Send Body As' is set to 'String'.

Property	Value
Output Queue URL *	
AWS Access Key Id	
AWS Secret Key	
Send Body As *	String
SQS Message Attributes	
Connection Timeout (ms) *	10000
Read Timeout (ms) *	30000

- **Output Queue URL** - The URL of the AWS SQS queue that this communication point will deliver messages to.
- **AWS Access Key Id** - The AWS access key identifier to use to send messages to the queue. This can be left blank if this is running in EC2 with an IAM role that grants the required permissions.

- **AWS Secret Key** - The AWS access secret key to use to send messages to the queue. This can be left blank if this is running in EC2 with an IAM role that grants the required permissions.
- **Send Body As** - Determines how the message body is sent. String means to load the message body as a string using its current character encoding, and transmit it as a string; Hex Encoded Binary loads the message body as a binary blob and transmits it using hex encoding; Base64 Encoded Binary loads the message body as a binary blob and transmits it using base64 encoding.
 - String
 - Hex Encoded Binary
 - Base64 Encoded Binary
- **SQS Message Attributes** - SQS message attributes can be set on the outgoing message using either literal values, or values retrieved from Rhapsody message properties. Prefix the value with \$ to indicate that a message property should be used. Attributes for message properties are only added if the message property is present and has a non-empty value. Each message can have up to 10 attributes.
 - Name
 - Value
 - Type: String / Number
- **Connection Timeout (ms)** - The connection timeout in milliseconds to use when attempting to connect to the Amazon SQS queue.
- **Read Timeout (ms)** - The read timeout, in milliseconds, to use when waiting for a response from an Amazon SQS queue.

Directory

Overview

Rhapsody can send and receive files from any accessible local and network directory using the **Directory** communication point. This communication point provides a wide range of management options, including the extraction of messages from a file (**de-batching**) or writing messages into an archive file (**batching**).

Read/Write Access

The Rhapsody user must have read/write access to all files it processes, otherwise the engine will not be able to pick up files from the input directory or save them to the output. If appropriate access is not available, the Directory communication point may appear to 'hang' as it attempts to process files, requiring a communication point restart before it begins to work correctly again.

Connection Modes

The Directory communication point supports all communication modes.

- Input
- Output
- Bidirectional
- In->Out
- Out->In

When Rhapsody has been configured to receive files from a directory, the directory is polled for new files at a fixed interval. As a result, any file placed in a directory that is being monitored by Rhapsody may wait for up to the refresh rate before being read from the directory by Rhapsody.

Rhapsody Associate Review

The use and configuration of the Directory communication point was described as part of the Rhapsody Associate course; it is recommended that you review your notes for this course before proceeding.

Configuration

The **Directory** communication point has a number of configuration options that depend on whether it is in Input or Output connection mode. These options are also available for the other connection modes but for the sake of simplicity, only these two modes will be described.

Input Mode Configuration Options

- **Input Directory** - The path to the folder holding the files to process. Rhapsody best practice suggests that a variable is used wherever possible in the configuration of an input (or output) directory as it keeps this information private, is easier to maintain and greatly simplifies a migration between environments.
- **Input Control File Mode** - Prevents a file from being processed until a second file with the same name, but different extension as specified in **Input Control File Extension** is added to the directory. This option is intended to be used with systems that generate a control file to indicate that a file is ready for processing.
- **Pattern** - Used to determine which files will be read from the directory for processing. The **Pattern Type** property selects between a **Wildcard** and a Regular Expression pattern type. When the wildcard * is specified as the pattern, all files in the input directory will be picked up for processing. The following examples illustrate how this option is used:

post* - a wildcard pattern that will process those files whose filename begins with the string 'post'.

*.edi - a wildcard pattern to process those files whose suffix is edi.

\d*\.\edi - a regular expression pattern that will process any file containing only digits in the base filename and with a suffix of .edi.

- **Exclude Pattern** - Defines the set of files to exclude from processing by Rhapsody. The default option for this property is <blank>, which means that no files are excluded from processing. The **Exclude Pattern Type** property selects between a Wildcard and Regular Expression pattern. The following examples illustrate how this option is used:

post* - a wildcard pattern that will exclude all files whose filename begins with the string 'post'.

*.edi - a wildcard pattern to exclude all files whose suffix is edi.

.*\d\.* - a regular expression pattern that will exclude any filename ending with one or more digits.

- **Refresh Rate** - Determines how frequently (in milliseconds) Rhapsody looks for new files in the specified directory.
- **Input Order** - When this property is set to None (the default), the Directory communication point will process files in the order they are read from the input directory. If the files in the input directory have an order, this property can be

changed to achieve this requirement. The settings of Filename (ascending or descending) or Least or Most recently modified first changes this behavior accordingly.

Selecting Custom JavaScript then populating the **Comparison Script** field with suitable code gives full control over the processing order, for example if there was a requirement to base the processing order on the files suffix. The Custom JavaScript option provides access to a Java list object which may be manipulated to provide the desired ordering.

- **Move Input Files** - The Directory communication point's default behavior is to delete files from the input directory once they have been picked up for processing. If the **Move Input Files** property is set to True and a **Processed Directory** specified, a copy of all files processed by this communication point will be stored in the specified directory for future reference. This may be used, for example, to maintain an archive of processed messages. **The Append Date/Time On Move** and **Append Date/Time On Move Format** properties can be used to add a date time to the moved file. This can could be used to avoid file naming conflicts on files in the processed filter.
- **Maximum File Size** - Specifies the maximum-sized file that Rhapsody will process. This option can be used to prevent from extremely large files from inadvertently being processed by Rhapsody. By default, this property is set to process all files.
- **Include Empty Files** - Allows files with a size of zero bytes to be processed. By default, empty files are not processed.

Output Mode Configuration Options

- **Naming Options** - The default behavior for files placed in an output directory is that their base filename is a timestamp with no suffix. This can be replaced with a hard-coded filename and suffix combination, with the former appended with either an incrementing number or a timestamp to avoid name clashes.

See below for a discussion on how a message property can be specified as a file's name and suffix.

- **Output Control File Mode** - When enabled, this option creates a control file in the output directory to indicate the file is ready for processing by an external system. When enabled, a control file can either be created with the same base filename but different suffix (filename.control), or same base filename and suffix, with the control suffix appended to the name (filename.suffix.control). The **Output Control File**

Extension defines the extension of the control file. By default, this option is disabled.

- **Output temporary file to** - When a route delivers a file to a Directory communication point in Output mode, that file is first written to a temporary directory specified by this property. Only when the file is fully written and verified will it be moved to the output directory specified in the communication point's configuration.

The location of the temporary directory can be changed by changing this property if, for example, the specified location is being used by another process.

Published Properties

A **Published Property** is a value extracted from an incoming message by the Rhapsody engine for use as a configuration option on the associated route. The published properties for a **Directory** communication point are:

- **BaseFilename** - everything before the '.' in the name of the file associated with the received message.
- **Suffix** - everything after the '.' in the name of the file associated with the received message.
- **FullFilename** - the combination of BaseFilename and Suffix.
- **FileCreationTime** - the time when the file was created. This property is specified as the number of milliseconds since 1 January 1970.
- **ZipBaseFilename/ZipSuffix/ZipFullFilename** - as above, but for zipped (archive) files associated with the received message.

Any of the above published properties can be specified in the configuration of the Directory communication point in Output mode. For example, if an incoming file was named ADT-56611420.edi and the communication point's **Base Filename** property is set to \$BaseFilename, the file saved to the output directory will be given the same name.

Other message properties associated with the message by, for example, the **Property Population** filter, can also be specified in the communication point's configuration using the above method.

Dynamic Folder Creation on Output

- Dynamic folder creation is available in Output mode by utilizing the following configuration options on the communication point
 - **Output Directory:** static, the path to the root (top) of the output tree; typically a Rhapsody variable.
 - **Base Filename:** dynamic, utilizing a property set on the route to provide a dynamic path under the root for output. For example, this might be set to include a date string and the filename to generate an archive of output messages separated by date – the value is treated as a literal, so any manipulation needed to set the path must be done on the route and stored in a property.

Batching and Debatching files

Depending on its connection mode, the **Directory** communication point can debatch (separate) incoming messages into their component files, and batch (combine) messages before saving them to an output directory. This is useful when the combined file is very large (containing many individual messages), which would place a large memory strain on Rhapsody if the combined message was placed on a route.

Debatch Mode

The Directory communication point in Input mode can to separate a file into its component messages by searching for a specified separator expression. This operation is also available using the more memory-intensive **Batch/Debatch** filter.

- **Debatch Mode** - This property must be set to True if incoming files are to be debatched.
- **Batch Header/Trailer Expression** - A regular expression that identifies the header or trailer string at the beginning or end of the batch file.
- **Separator Type**- Sets whether the message separator separates messages or marks the start of a message.
 - Marks Start of Message - identifies the string that marks the start of the message, and which is part of the message itself.
 - Separates Messages - identifies the string (specified in the Message Separator Expression field) found between messages. This string is not part of the message.

- **Message Separator Expression**- The character(s) identifying the start of each message or separates messages from each other. When Separates Messages is selected, the appropriate escape sequence for the end of line characters for the message must be specified. For example:
 - HL7 messages: \r
 - DOS messages originating from a Windows PC: \r\n
 - Unix Linux: \n

The Directory communication point also includes the **Zip File Debatch Mode** property which, when set to True, will separate the components of an incoming zip (archive) file into individual files. If this option is selected, the option to debatch incoming messages will not be available.

Batch Mode

The Directory communication point in Output mode can combine processed messages into a single file using a specified wrapper to identify the beginning and end of each file.

- **Batch Mode** - This property must be set to True if outgoing files are to be batched
- **Outgoing Wrapper**- Sets whether the message separator separates messages or marks the start of a message.
 - None - no message wrapping is used.
 - Minimal - the wrapping uses a header of \x0B and a trailer of \x1C\x0D, as specified by the HL7 Minimal Lower Layer Protocol (MLLP).
 - User - uses custom header and trailer strings. The endian-ness of binary data that can be placed in these header/trailer strings is set in the Endian-ness properties. These endian-ness properties can be either Big (most significant byte first) or Little (least significant byte first) endian.
- **Outgoing Header/Trailer** - (only available if Outgoing Wrapper is set to *User*). Sets the header or trailer written to the file before or after each message
- **Outgoing Endian-ness** - (only available if Outgoing Wrapper is set to *User*). Sets whether or not the most significant byte is first or last when binary data is included in the header and footer strings
- **Batch File Rollover** - Identifies the frequency with which batch files are rolled-over. Options: Size (user defined), Hourly, Daily, Weekly, Monthly, Never.

- **Batch File Rollover Size** - (only available if Batch File Rollover is set to Size). Sets the maximum size to which the batched file can grow before being saved and a new file opened.

The Directory communication point also includes the **Compress Batch File** property which, when set to True, will be saved in .zip format, reducing its final file size. If this and the **Batch Mode** properties are set to True, there will be one entry per message and the wrapper header and trailer properties will be ignored. If False, all messages will be in one entry inside the zip file.

ZIP files corruption

A ZIP file created by the directory communications point will not be valid on disk until after rollover occurs. If Rhapsody loses access to the file. For example, if the file is on a network directory that is disconnected, or if the Rhapsody engine crashes, the ZIP file will remain invalid permanently and the contents will not be accessible. For this reason, it is recommended that the ZIP filter (discussed in *Applying Business Logic*) is used when files need to be compressed.

Ensuring Files are Complete

When reading files from a directory, it is important that Rhapsody only reads the file for processing at the point when it has been completely written to the directory. Likewise, any system reading in files generated by Rhapsody must only attempt to read the file once the file has been completely written to disk.

Rhapsody has two methods for ensuring that this is the case:

- Temporary files
- Control files

The two methods are complementary and can be used together if required.

Temporary files

Temporary files are the default method used by Rhapsody to ensure only complete files are visible. When outputting a file, Rhapsody writes the file to either a temporary directory or a file with .tmp extension (as determined by the **Output temporary file** to configuration property). Once the file write is complete, Rhapsody renames the file to the intended filename. The destination system needs to be configured to ignore the temporary file.

In input mode, Rhapsody assumes that any source system has used a similar process to write files to the directory. The **Pattern** and **Exclude Pattern** properties can be used to ensure that any temporary files that might exist are not processed.

Control files

A **Control File** (also known as an **OK File**) is a file whose presence indicates that an associated data file is complete and/or is ready to be processed. The presence of a control file signals to a remote system or Rhapsody communication point in Input mode indicates that the data file is ready for processing. A remote system or Rhapsody communication point in Output mode will only create a Control File after the data file is completely written.

Input Control File Mode

When a Directory communication point's **Input Control File Mode** property is True, the presence of a control file in the input directory indicates that its associated data file is ready for processing. Both files will be supplied by an external system.

Rhapsody recognizes a control file by its extension, as defined by the (required) **Input Control File Extension** property. If this is set to, for example, 'ok' and its associated data file is named 'patientdata.xml', Rhapsody will initiate processing if a control file named either patientdata.xml.ok or patientdata.ok is present in the input directory. Rhapsody also supports the case where the data file does not have a suffix. For example, it is named patientdata. In such a case, the generated control file will be named patientdata.ok.

Note that the Input Control File Extension property is case-sensitive.

Output Control File Mode

When a Directory communication point's **Output Control File Mode** property is True, the communication point will create a control file only when the entire data file has been written to disk. This signals to an external system that the data file is complete and can be picked up for processing.

Rhapsody generates a control file using the extension defined in the **Output Control File Extension** property. This is applied in either of two ways:

- appended to the existing filename; for example myFile.csv → myFile.csv.ok
- replacing the existing filename suffix; for example myFile.csv → myFile.ok

The use of a Control file is an alternative to relying on Rhapsody saving output files to a temp directory and only moving them to the output directory once they have been completely written.

Network Directories

By default, when Rhapsody is installed on Microsoft Windows, the Rhapsody service will use the *Local System* account to run the Rhapsody Service. This account does not have access to any network drives. If a directory on another machine needs to be accessed, the account used to run the service will need to be changed to an account with suitable permissions both locally and on the other machine.

When choosing an alternate account to run the Rhapsody Service, ensure the account has sufficient privileges to access all required local files. In particular, ensure the account has full control of all Rhapsody files, including the datastore.

UNC Naming

UNC (Universal Naming Convention) paths are supported for identifying a directory on a remote machine, provided the Rhapsody server is a Windows host and the Rhapsody engine runs as a Windows domain user. The specification for a UNC path normally will match that for the share: \\server\share\path_to_file.

Changing the Rhapsody Service Account

Network access credentials are typically specified during the Rhapsody engine installation. They may be specified or changed at any other time with the following steps:

1. Select **Start** and search for **Services**.
2. 2

Select the **Services** app from the search results.

3. 3

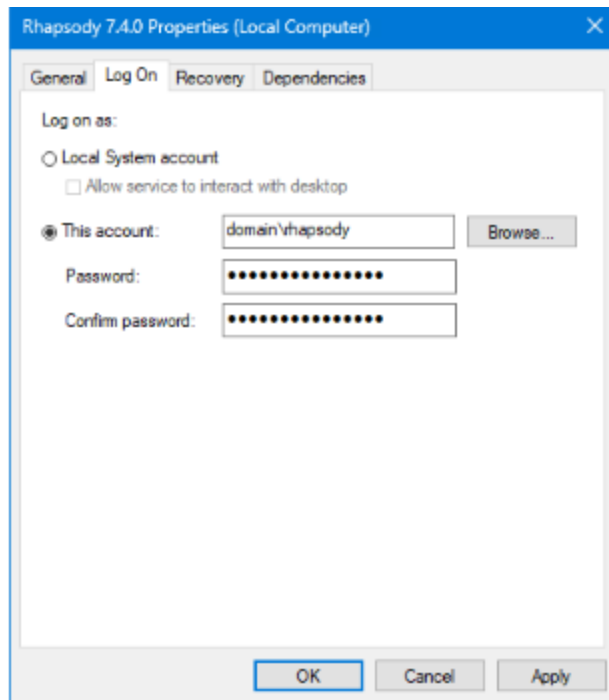
Scroll down and select the **Rhapsody** service (the actual service name will include the Rhapsody version number).

4. 4

Right-click and select **Properties**.

5. 5

Select the **Log On** tab; an image similar to the following will be displayed:



1. Provide appropriate login credentials.

2. 7

Select **OK** to save the new credentials.

3. 8

Restart the Rhapsody Engine to apply this change.

S)FTP Client

Overview

The **(S)FTP Client** communication point is used to connect to an **FTP Server** to fetch files from an input directory, or send messages to an output directory, or both. It supports the following transfer protocols:

- FTP
- FTPS (FTP over SSL)
- SFTP (SSH File Transfer Protocol)

This communication point can be configured statically or dynamically at runtime, providing multiple connection management. These concepts will be discussed in this module.

The **(S)FTP Client** communication point supports Input, Output and Out->In modes only.

Do not configure multiple **(S)FTP Client** communication points to access the same file in an input directory. This could cause a race condition if simultaneous connections attempt to read the same file. Note that if it is just one communication point that has spawned multiple connections, that communication point will handle possible race conditions, and blocking will be handled by the communication point where necessary.

FTP vs SFTP vs FTPS: What's the difference?

When deciding which FTP connection type to choose, you often do not have to look any further than the server admin and simply ask what protocol type the server supports. A more challenging scenario would be when the FTP server connection type is not determined and has to be decided on by the team. So, what's the difference between these three protocols?

First, the commonalities: All three protocols provide the ability to upload, download and delete files, create and delete directories, and read directory contents.

FTP

FTP was the original protocol for file transfer over networks. Due to its popularity, it is in frequent use today around the world. Although one may initially gravitate towards a basic FTP protocol connection type in Rhapsody, it is important to note a few disadvantages when compared with the SFTP and FTPS connection types:

- Presence of a secondary data connection: Although this occurs behind the scenes, it is a limitation in the protocol. The two channels are the command and data channel. These channels are unencrypted and makes communication more difficult behind firewalls.
- No security protocol when in default mode: Please see FTPS for how we can employ a security layer on the FTP protocol.

FTPS

We can think of FTPS as the FTP protocol with a security layer; Specifically, the SSL/TLS protocol. This defines channel encryption for the FTP server. Yet, although FTPS is widely used, it still has the same disadvantages of the FTP protocol it is based on, namely the presence of a secondary data connection. This is also sometimes referred to FTP over SSL. Authentication is conducted using X.509 certificates.

SFTP

SFTP isn't as it sounds. This protocol is based on the Unix SSH (Secure SHell) family of protocols and not the FTP protocol discussed above. SFTP is sometimes mistakenly referred to as "Secure FTP". It is important to note that, although secure, SFTP is based on the SSH protocol. SFTP is actually shorthand for SSH File Transfer Protocol. Authentication is conducted using SSH keys. A system that supports shell access via SSH will also support connections via SFTP.

SFTP has a few advantages over the standard FTP (FTP, FTPS) based protocols:

- Connection is always secured.
- Provides more file-system operations (file lock, symbolic link creation).
- Communication is over one channel, unlike FTP/FTPS two channel communication.

We recommend that for overall flexibility of protocol, SFTP should be used if given the choice. Although there may be overhead in understanding and managing SSH keys, it is worthwhile to explore this protocol due to the advantages noted above. That said, both FTPS and SFTP deliver strong encryption and both should be considered when moving files to and from servers.

Configuration Approaches

The (S)FTP communication point can be configured in one of three different ways using the **Configuration Mode** property depending on messaging requirements:

Method	Description
Use Static Configuration	The communication point is configured by directly entering configuration, message properties and environment variables references into the properties window for the communication point.
Dynamic Using XML Configuration Template	Using an XML file to specify one or more connection profiles. A message property is used to determine which profile is used for a given message.
Using Message Property Configuration	Configuration specified in message properties overrides static configuration. When configured this way, all necessary static properties should be defined.

The configuration properties required to configure the communication point are the same regardless of the mode selected. The only difference is the method used to specify the properties.

Dynamic Mode

The **Dynamic Using XML Configuration Template** and **Using Message Property Configuration** methods are collectively referred to as Dynamic Mode. These modes are used when a single communication point needs to connect using a range of different connection settings and using two communication points is not practical.

When running dynamic mode, the communication point can only be run in Output or Out->In modes.

Dynamic Output Mode

The (S)FTP Client communication point is able to operate in a dynamic output mode that enables its configuration parameters to be changed dynamically at runtime. The outgoing message determines the configuration settings that should be used to send, even down to the FTP protocol to use, in other words it is possible to dynamically switch between the FTP, FTPS and SFTP protocols.

When operating in dynamic output mode, the (S)FTP Client communication point's configuration parameters can be changed dynamically at runtime. Configuration settings configured statically in the Rhapsody IDE are merged with configuration parameters

retrieved from either the appropriate configuration template, or the message properties of the outgoing message.

The communication point is placed into dynamic output mode by setting its **Mode** to *Output*, and then setting the Configuration Mode property on the communication point to either Dynamic Using XML Configuration Templates or Use Message Property Configuration. Once this is enabled, all configuration properties are enabled on the communication point, even those that would normally be disabled. This is to allow appropriate default values to be configured for any desired configuration properties, even though some of these defaults will not be used simultaneously. For example, once in dynamic mode it is possible to configure both SSL certificates and SSH public keys to use to validate the remote server, but obviously at most one of these will be used at a time.

Dynamic Input Mode (Out->In)

The purpose of using dynamic input is to allow a single communication point to be able to dynamically retrieve messages from a variety of different remote servers as required based on an event. This replaces the need to have an input communication point configured for every single remote server that you are connecting to. It also means that you can set up your workflow to only connect to remote servers as required, rather than relying on polling or scheduling.

To enable dynamic input, the communication point should be configured to be operating in the Out->In mode, and **Configuration Mode** set to either Dynamic Using XML Configuration Templates or Use Message Property Configuration. To trigger dynamic input retrieval, a message is sent to the output of the Out->In communication point (henceforth referred to as the trigger message). The body of the trigger message is unimportant as this message itself will be discarded, but the properties of the message are important. Using the message properties, you can set the input side configuration by either selecting the appropriate configuration template or providing values for all the appropriate configuration settings (for example, server, port, SSL/SFTP settings) in the same way that you would for dynamic output. In the same way as dynamic output, these settings will overwrite the static configuration if set, but otherwise the static configuration will be defaulted to. This allows parameters that are common across remote servers to just be configured statically. Note that Out->In mode can be used with an entirely static configuration as well if required.

The **Refresh Rate** cannot be set dynamically, as it is irrelevant when operating in this mode because Rhapsody does not poll for input.

If no input is available from the remote server when a trigger message is sent to an Out->In communication point, the trigger message will be discarded by default and the next trigger message on the queue will be sent. This will appear in the message path of the management console. However, if the **Generate Message When No Response** configuration property is enabled, the (S)FTP Client Communication Point will instead return a single dummy response message if no files are available for download. This dummy response message can be identified by the presence of a message property called `NoResponseMessageReceived`, and allows the Rhapsody route to perform any necessary logic to handle a server that currently has no files available for download.

If multiple input messages are available from the remote server when the trigger message is sent, then the trigger message will be re-enqueued to the front of the queue and resent. This means that all the available input can be received from that remote server before the next trigger message is received. This will appear in the message path of the management console, and as is the usual behavior with Out->In mode, all the received messages will show in the message path too. This makes it possible to see exactly what input any given trigger message caused to be received. The last message received for each trigger message will have a message property called `LastResponseMessageReceived` set, which allows the Rhapsody route to perform any extra processing that may be required once all files have been downloaded.

Defining the Server

The following properties define how the FTP communications point connects to the FTP server.

- **Server** - Hostname/IP Address of the FTP Server to which the communication point will connect.
- **Port** - the remote port number to connect to on an FTP server. The default value is 0, which means port 21 is used for all standard FTP connections (port 990 is used when running in FTPS implicit mode).
- **Username** - the username/id of the user for authenticating with the FTP server.
- **Password** - the password of the user for authenticating with the FTP server (the user account must have write access).
- **Read Timeout** - specifies the length of time Rhapsody will wait for a response from the FTP server (in milliseconds).
- **Transfer Mode** - Options: **Binary** or **ASCII**. Default: **Binary**. Identifies the mode transfer type to use. In Binary mode, the sending machine sends each file byte for

byte and as such the recipient stores the byte-stream as it receives it (The FTP standard calls this "IMAGE" or "I" mode).

When a file is sent using an ASCII-type transfer, the individual letters, numbers, and characters are sent using their ASCII character codes. The receiving machine saves these in a text file in the appropriate format (for example, a Unix® machine saves it in a Unix® format, a Windows machine saves it in a Windows format).

In ASCII mode, any form of data that is not plain text will be corrupted.

- **Connection Mode** - Options: **Maintain a constant connection** or **Disconnect if not sending/receiving**. Default: **Maintain a constant connection**. This identifies the type of connection maintained with the FTP server. A constant connection is not advised if the communication point is expected to stay idle for extensive periods of time.

Published Properties

Like many communication points, the (S)FTP communication point create a number of message properties that describe messages received from a server when running in either Input or Out->In modes. The properties specific to the (S)FTP Client communication point are:

- **FullFilename** - the full filename of the file associated with the received message.
- **BaseFilename** - the base filename of the file associated with the message.
- **Suffix** - the suffix of the file associated with the received message.
- **InputDirectory** - the directory path that the file associated with the received message

Specifying Files

Input Mode Properties

The following properties define how the (S)FTP communications point receives files from the server.

- **Input Directory** - the directory on the server to check for input files. If empty, the user's home directory becomes the Input Directory.
- **List Command Parameter** - the List Command parameter to pass to the remote server. This is only needed if the server you are connecting to expects different commands to the standard ls command. Generally, you should not have to set this property.

- **Input Pattern & Input Pattern Type** - pattern to use to select input files from the Input Directory. These properties work the same way as their equivalents on the Directory communication point.
- **Input Order & Comparison Script** - determines the order the messages will be read from the server. These properties work the same way as their equivalents on the Directory communication point.
- **Refresh Rate** - specifies the frequency (in ms) that the input directory is checked for new files. This property is optional, with a default value of 60000ms (1 minute).
- **Remove Files from Server** - Options: True or False. Default: True. This option controls if the files should be removed from the server after being successfully downloaded. This option supports cases where another process is responsible for removing files and therefore, should almost always be set to True. If set to False, the route must be able to handle any duplicate messages from this communication point.
- **Maximum File Size for Download (MB)** - any file over the specified size is skipped and not downloaded. This property can be configured statically, dynamically or via configuration templates.

Output Mode Properties

The following properties define how the (S)FTP communications point sends files to the server.

- **Output Directory** - the directory on the server to place output messages in. Write access is required. The default will be the home directory if empty.
- **Base Filename** - Base filename to use for output messages.
- **Suffix** - Optional suffix to append to the end of the filename.
- **Append Date/Time** - Options: True or False. Default: True. Appends the current date/time to the filename. Note that the default is True; this compares with the default: False for the Directory communication point. This difference is because it is harder to test for unique names over FTP and SFTP. This makes it more important to generate a unique file name.
- **Append Date/Time Format** - Options: Defines the format of the date/time appended to the filename.

- **Check Name** - Options: True or False. Default: True. If False, existing files with the same name as the generated file will be overwritten. If True and a duplicate filename is encountered, a counter will be added to the newly generated filename. Setting this property to True significantly slows the performance of the communication point.
- **Use temp file** - Options: True or False. Default: True. If True, a temporary file is created on the server and renamed once transmission is complete. This option helps ensure that any processes reading the file from the server will only see the file once it has been completely written to the server.
- **Append to existing file** - this will allow the transfer process to append messages to a file that may already exist on the remote server. This property cannot be set to true if any of the following options are set to true:
 - Use Temp File
 - Check Name
 - Append Date

Secure Server Properties

The following properties define how the connection to the FTP server is secured (when required).

Common options

- **Protocol** - This determines the protocol being used to connect to the server. The FTPS Implicit and FTPS Explicit options represent two different ways a FTPS server can support secure connections.
 - FTP
 - FTPS Implicit
 - FTPS Explicit
 - SFTP
- **Server Validation Mode** - This is used to determine how the (S)FTP Client communication point finds the SSL certificates or SSH public keys for server validation. This is done prior to logging in.
 - Use Preconfigured Keys

- Allow Dynamic Key Lookups
- Auto-Add Keys For New Servers
- **Cipher Suites** - Configures the level of security Rhapsody will use when selecting ciphers for the secure connection.

FTPS options

When the **Protocol** has been configured as either **FTPS Explicit** or **FTPS Implicit**, the following properties need to be considered so that a connection to the server can be made.

- **SSL Server Certificates** - This selects the certificates used to authenticate the FTPS server. The Server Certificates are selected from the certificates configured in the Rhapsody IDE Certificate and Key Manager (Refer to the *Rhapsody Services* modules for details on managing certificates).
- **Hostname Validation** - Determines whether the SSL certificates presented by the FTPS server should be validated to ensure they are for the correct server.
- **FTPS Options** - Provides options to work around some known bugs in various FTPS servers. A number of FTPS servers do not correctly close SSL channels when closing either the data or control sockets. This can manifest itself as either the client communication point freezing until a timeout occurs, or raising connection errors when attempting to close data or control sockets. If this occurs, then the appropriate options can then be enabled. Note that these options should not be enabled unless required to fix a specific FTPS server issue otherwise as they instruct the FTP communication point to use non-standard SSL closure mechanisms.
 - Disable control SSL closure
 - Disable control wait on close
 - Disable data SSL closure
 - Disable data wait on close
 - Disable SSL session resumption
 - Start with clear data channels
- **SSL Client Private Key** - The SSL private key is used for client authentication when connecting to an FTPS server. The SSL private key is selected from the keys configured in the Rhapsody IDE Certificate and Key Manager.

SFTP Options

When the **Protocol** has been configured as **SFTP**, the following properties need to be considered so that a connection to the server can be made.

- **SFTP Client Authentication Mode** - This configures what type of SSH client authentication is performed when connecting to an SFTP server.
 - Password Authentication
 - Public Key Authentication
 - Password and Public Key Authentication
- **SFTP Client Private Key** - This selects the single SSH private key used for client authentication when one of the public key authentication modes is selected. It must be configured when public key authentication is enabled; it is not available otherwise. During client authentication, the client transmits the SSH public key associated with this private key to the SFTP server, along with proof that it is the holder of the private key. The SFTP server can then match the SSH public key against its configuration in order to authenticate the client. The private key itself is never transmitted to the server. The SSH Private Key is selected from the keys configured in the Rhapsody IDE Certificate and Key Manager (Refer to the *Rhapsody Services* modules for details on managing keys).
- **SFTP Compression** - Determines which compression algorithms the (S)FTP communication point advertises its support for.
- **SFTP Advanced Configuration Flags** - Provides options to allow the (S)FTP Client Communication Point to successfully operate with some known bugs in various SFTP servers.
 - Disable chmod after creating a directory
 - Disable chmod after uploading a file
 - Disable wait for channel close
- **SFTP Server Public Key** - Holds a list of the SSH public keys used to authenticate the SFTP server. The SSH Public Keys are selected from the keys configured in the Rhapsody IDE Certificate and Key Manager.

Custom Commands

Custom commands are specific configuration properties that represent custom (S)FTP commands. These commands are sent to the server and executed before or after sending/receiving a message.

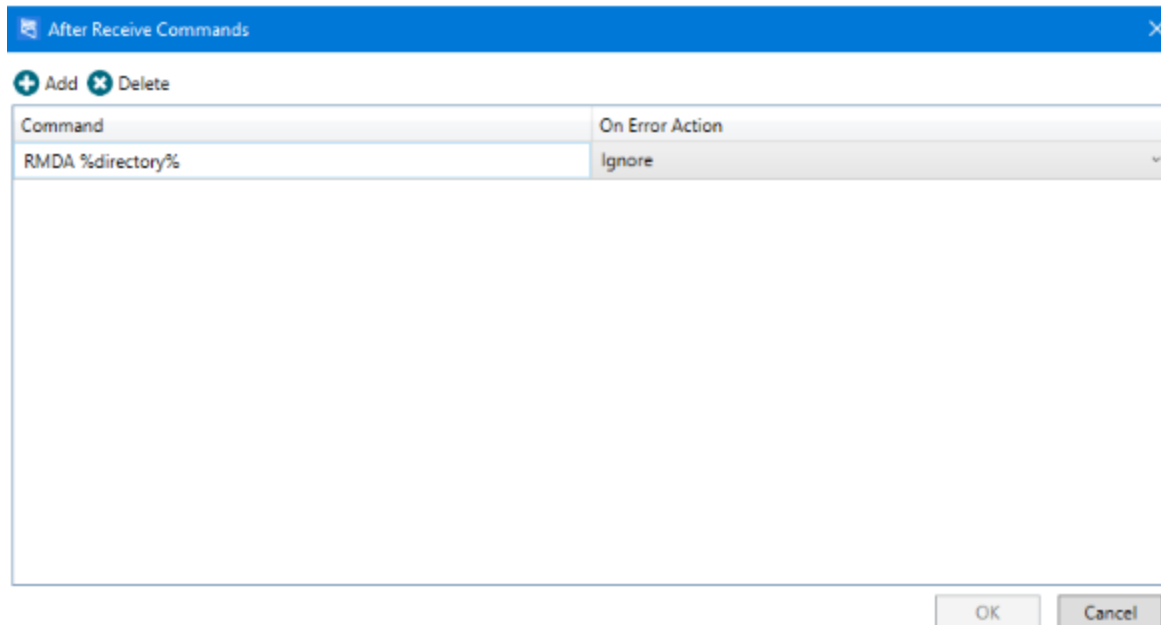
Commands can use Rhapsody variables by enclosing the variable name in '%', for example %variableName%. Two variable examples would be to reference the directory or filename as the following:

%directory%

%filename%

You are able to have multiple commands. These commands will be executed sequentially from 1..n, where n is the total number of commands. Commands are protocol, site, and environment specific.

Command	On Error Action
OPTS UTF8 ON	Treat as Connection Error
LPSV	Blacklist File



On Error Action

The **On Error Action** setting for each command defines the action to take if the command fails. There are three options for this field:

- **Ignore** - the error is logged but ignored, so processing continues normally.
- **Treat as Connection Error** - the error is treated as a connection error and so the normal connection retry functionality takes effect.
- **Send to Error Queue** - the outgoing message is sent directly to the Rhapsody error queue. This option is only available for Before Send Commands and After Send Commands.
- **Blacklist File** - the incoming message is blacklisted (ignored) temporarily as though it failed to download. This option is only available for Before Receive Commands and After Receive Commands. Blacklisting is only temporary and the file is tried again later.

Custom commands sent to an FTP(S) server are treated as successful if the FTP server returns a code of 1xx, 2xx, or 3xx. The command is considered to have failed if any other type of response is received. Custom commands sent to an SFTP server are different, as the server does not return the same sort of error code. Consequently, custom commands on an SFTP server are treated as failed if the server reports an error to the client while invoking it, or if the command writes to standard error.

References

- http://en.wikipedia.org/wiki/List_of_FTP_commands(opens in a new tab)

Archiving

Where the (S)FTP communication point is used to receive many large messages from an external system, the Rhapsody archive may grow to a very large size. This can be controlled with the **Archive Message Bodies** property. The two options are:

- Normal Archiving Strategy (default)
- Cleanup After Sending Message

If this communication point is working with large files and the Rhapsody archive size is of concern, then the Cleanup After Sending option should be considered. This will cause messages to be deleted from the archive as soon as they are sent from Rhapsody.

Organizational policies on archiving data may need to be considered before utilizing this feature.

Configuration with Message Properties

When the Configuration Mode of the communication point is set to **Use Message Property Configuration**, one or more message properties can be set against a message sent to the communication point to override settings made in the communication point.

Details of the available message properties can be found in the documentation:

[Open docs](#)

Tips For Using (S)FTP Communication Point Message Properties

While you can read through the specific properties names and acceptable values on the documentation page noted above, we feel it is worthwhile to review a list of tips for using message properties with this communication point.

1. If you change a property value of a communication property, this will cause the connection of the communication point to be re-established.
2. If setting a property, please ensure the value is not invalid or an empty string; If so, the message may end up on the error queue if the value is required and not statically configured in the communication point.
3. When you specify a port value, '0' represents the default value based on the protocol. For example, if '0' is specified for SFTP port, then the value 22 is used behind the scenes.

4. If a Read Timeout value of less than 1000ms is provided, an error will occur on the communication point.
5. Some configuration values are ignored depending on the protocol (SFTP, FTP, FTPS); Please pay attention to the protocol you are using.
6. You can set multiple property values using property lists. These lists are denoted by '[...,...]' where each value would be separated by a semicolon. Note that only certain properties are in list format.
Refer to the documentation for specific properties that accept list values.
7. When referencing certificates, always ensure they are loaded in the Certificate and Key Manager in the Rhapsody IDE.
8. If a property value is set, it will override the value in the configuration properties.

In summary, the basic flow for using configuration properties is the following:

Use the property value only if it is a property that is valid for the protocol type (SFTP, FTP, or FTPS). If valid, use the property value if it is not empty. If it is empty, an error will occur and the message will be sent to the error queue only if the corresponding static value is empty. The exception to this is for Username and Password, as empty strings are accepted for both.

When should Message Properties be used?

If there is any type of dynamic nature to a property, please utilize the corresponding message property. For more detailed scenarios using message properties, please refer to the next section on Configuration with an XML Template.

Configuration with an XML Template

Think of an XML Template as a set of properties for the (S)FTP communication point, coded in XML, that can be selected at runtime. This is simply a collection of properties, just as one would configure using message properties, but stored as a configuration set using XML. A property value on an outgoing message can be used to determine which XML template file to use.

There are various child nodes of the <Template> node. Each of these nodes represents a configuration property for the communication point.

The XML is specified via the **Configuration Templates** property.

Simple XML Example

```
<?xml version="1.0"?>
<ConfigurationTemplates>
  <Template id="one">
    <ConnectionType>SFTP</ConnectionType>
    <ServerHostname>$(server1)</ServerHostname>
    <ServerPort>22</ServerPort>
    <SshServerPublicKey>sshPubKey1</SshServerPublicKey>
    <SshClientAuthMode>password</SshClientAuthMode>
    <SshCompressionMode>zlib@ssh.com</SshCompressionMode>
    <SshConfigFlag>disableWaitOnChannelClose</SshConfigFlag>
    <SshConfigFlag>disableChmodOnMkdir</SshConfigFlag>
    <SshRemoteEol>unix</SshRemoteEol>
  </Template>
  <Template id="two">
    <ConnectionType>SFTP</ConnectionType>
    <ServerHostname>$(server2)</ServerHostname>
    <SshServerPublicKey>sshPubKey2</SshServerPublicKey>
    <SshClientAuthMode>publickey</SshClientAuthMode>
    <SshClientPrivateKey>sshPrvKey1</SshClientPrivateKey>
  </Template>
</ConfigurationTemplates>
```

The example here defines two templates with the identifiers one and two. Each template is required to have a unique identifier. A template is then selected by an outgoing message by setting a message property called TemplateId to the template identifier. If the selected

template cannot be found, or if no template is specified for a message, then the outgoing message will be sent to the error queue.

Communicating with Other Rhapsody Connections

Overview

The communication points described in this section communicate with other Rhapsody connectors either within their own environment or, in the case of the Rhapsody Connector, another Rhapsody engine.

Sink

The **Sink** communication point immediately discards all messages it receives. It is used for messages that do not require further action, such as a positive acknowledgement. This communication point is only available in Output mode.

Despite having been discarded, the content and associated properties of all messages sent to the Sink can still be viewed by opening the Rhapsody message archive. The length of time these remain in the archive is configured by the site Administrator. By default, messages are retained for 28 days, after which they are purged.

Timer

The **Timer** communication point is used to trigger an event in the Rhapsody engine at regularly occurring intervals. The communication point is only available in Input mode and has two configuration options:

- **Body Template File** - the static file whose injection into the route triggers the desired event
- **Refresh Rate** - the frequency (in milliseconds) with which the file is sent into the route

For example, assume an administrator wishes to know the number of messages held in an external database at different times of the day or week. This can be achieved by configuring the **Timer** communication point to send a file to a **Database Message Extraction** filter, which is then utilized to extract this information from the database in XML format. The XML file can then be converted to PDF, attached to an email and sent to the administrator.

The options on the **Scheduling** tab of the Timer's **Properties** screen can be used to further control the times at which the desired event is triggered.

Route Testing

An alternative use of the Timer communication point is to test a route's ability to handle many messages sent at short time intervals. One disadvantage relating to this use, however, is that the communication point's configuration allows only one file to be sent, which is not a realistic test for most environments.

Watch List Notifier

A **Watchlist** is a list of engine events that an administrator has grouped and associated with a list of recipients who can be notified when a monitored event occurs. Typical notifiable events include:

- A message has been sent to the Error Queue
- One or more routes and/or communication points have been idle for longer than a pre-set time
- A web service has shut down unexpectedly

Watchlists will be discussed in more detail in *Managing the Rhapsody Environment module*. The watchlist notifier receives messages sent to a watchlist. This would be used if a group of users needs to be notified of specific engine events via a communications mechanism not supported by the notifications system (for example, an HL7 message or database).

The **Watch List Notifier** communication point is configured to listen for events sent to one or more watchlists. When a message is sent to a watchlist, the message is also received by the communications point. Each message received by the watch list notifier will be an XML message that describes the notification event:

```
<event id="926" issueTypeId="402849797"
name="MESSAGE_SENT_TO_ERROR_QUEUE_BY_ROUTE">
```

```
<displayname>
```

```
Error Queue Message
```

```
</displayname>
```

```
<severity escalation="false">
```

```
WARNING
```

```
</severity>
<deliverytype>
  NORMAL
</deliverytype>
<enginehost>
  Rhapsody
</enginehost>
<timestamp>
  2012-11-29T11:13:06.495+13:00
</timestamp>
<initialtimestamp>
  2012-11-29T11:13:06.495+13:00
</initialtimestamp>
<component id="1866" guid="589cd095-e7ff-45bf-9048-f5711284c4c8">
  <name>
    Check for PatientId
  </name>
  <path>
    PASTastic\Update PatientId
  </path>
</component>
<parameter id="1">
  <displayname>
    Message Identifier
  </displayname>
  <value></value>
```



```
</parameter>  
</event>
```

Message Tracking

Overview

Message tracking is used in Rhapsody to ensure that messages sent to an external system are being processed correctly, and that error conditions are managed correctly.

Two approaches exist in Rhapsody for tracking messages:

- Implicit tracking using a communications point in either In->Out or Out->In mode.
- Explicit tracking using a Message Tracking Scheme on a communications point in Out->In or Bidirectional mode.

This lesson will cover both approaches.

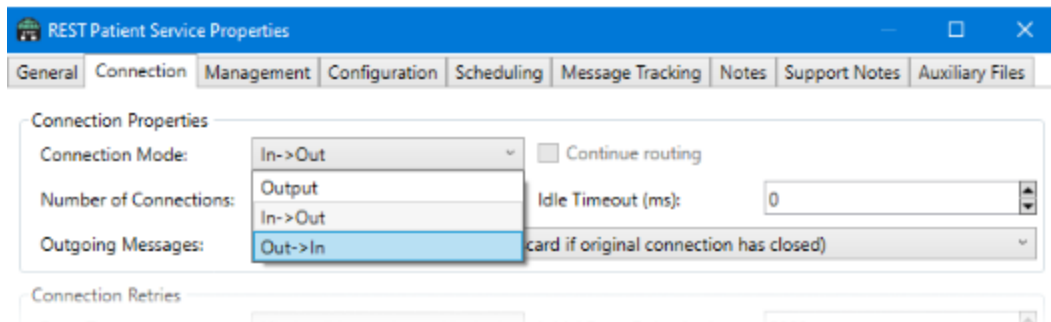
Example usages

Scenarios where message tracking is useful:

- Tracking of outbound messages and any corresponding error response message (including options to automatically stop the communication point).
- In Out->In mode, recovery from a situation where Rhapsody sends a message to the communication point, then waits for the external system which has timed out and not replied with a response.
- When the external system is reporting a temporary fault (for example, the external system cannot connect to its database) and that re-sending the message may succeed.

Communications Point Mode

Implicit tracking takes place when the connection mode for the communications point is set to either In->Out or Out->In.



In In->Out mode, Rhapsody waits for a message to be received from the communications point. It processes the message and sends a response to the same communications point before receiving the next message. This mode is used when we want Rhapsody to only process a single message per connection from an external system.

When operating a communication point in In->Out mode, care needs to be taken to ensure every received message generates an output message to the same communication point. If the number of messages received by the communication point does not match the number of messages sent to the communication point, it is possible for some messages to be delayed. When the number of messages received exceeds the number sent, receipt of additional messages will be delayed. When more messages are sent then received, there will be a queue of messages in Rhapsody waiting to be sent. Care also needs to be taken to ensure that message processing within Rhapsody does not take an excessively long time. This will delay the response message and processing of subsequent messages.

Where possible, it is often better to use Bidirectional mode in preference to In->Out mode.

In Out->In mode, Rhapsody sends a message from the communication point, then waits for the external system to return the response to the same communication point before sending the next message. This mode is normally used with an external system that can only safely process a single message at a time or to preserve messaging requirements (for example, the HL7 message / ACK requirement).

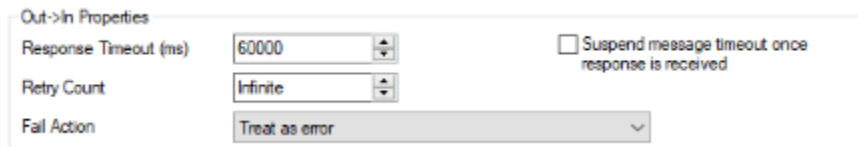
Both In->Out and Out->In modes have a number of properties that control how the communications point behaves when an error occurs. These define how long Rhapsody will wait for a response message and what actions should be taken when a response message is not received.

In->Out Properties



- The **Response Time** property determines how long Rhapsody will wait for the response message to be sent to the communications point before resetting and accepting the next incoming message. Where a sending system resends messages after a timeout, the value of this setting should be less than the sending system's resend timeout value. A value of zero will result in an infinite timeout. An infinite timeout should not be used as messages that do not generate a reply (such as those sent to the error queue) would prevent further messages from being received.
- The **Discard message once timeout occurs** property, when selected, sets a time-to-live equal to the **Response Timeout** on each incoming message. This prevents any additional processing being performed on a message which would be sent to the Communication Point after the **Response Timeout** has occurred.

Out->In Properties



Out->In Properties

Response Timeout (ms)	60000	☐ Suspend message timeout once response is received
Retry Count	Infinite	
Fail Action	Treat as error	

- The **Response Time** property determines how long Rhapsody will wait for the response message from the external system. Where a receiving system has a timeout value, this setting should be set to a value greater than that system's timeout value. An infinite timeout will prevent Rhapsody from resetting to output mode, and should only be used if you can guarantee that the external system will eventually send a response message.
- The **Suspend message timeout once response is received** property disables the **Time-To-Live** for a message received by the Communication Point. This ensures that processing of a response from a server completes, even if it took longer than expected for the response to be received.
- The **Retry Count** property determines how many times Rhapsody will attempt to resend the message if a response is not received within the response timeout period.
- The **Fail Action** property determines what happens to a message not received after the number of retries specified above.

Summary of settings

The table below summarizes the various communication point behaviors when sent and received message volumes do not match.

Communication Point Mode	Received > Sent	Sent > Received
In->Out	Receipt of new messages by Rhapsody will be delayed. This can occur due to one of the following reasons: • When some messages don't generate a response that is sent outbound to the communication point. When a message generates an error, remember that a response still needs to be generated and sent. • When the sending system has a short timeout and resends a message before the reply has been generated	There will be a queue outbound on the communication point. This happens as a result of a configuration that is sending more messages to the communication point than are being received. A change to the configuration will be needed to ensure that the number of sent and received messages always match.

Communication Point Mode	Received > Sent	Sent > Received
	in the Rhapsody configuration. The Response Timeout property determines how long Rhapsody will wait before resetting the communication point to input mode to receive the next message.	
Out->In	Depending on the behavior of the external system, this can result in acknowledgement messages being matched with the wrong output message. This can occur due to a fault in the external system (causing it to send additional responses), or a timeout. Switching to a message tracking scheme will ensure that each acknowledgement is matched with the	A queue of unsent messages will appear outbound on the communication point. This frequently happens when an external system is not able to process messages as fast as they are being sent by Rhapsody. It can also occur when the external system occasionally fails to respond to a message. The Response Timeout and Retry Count properties determine how long Rhapsody will wait before attempting to resend the message or moving to send the next message. When the condition is caused by temporary high message volumes, the communication point's output rate limiting settings can help.

Communication Point Mode	Received > Sent	Sent > Received
	correct output message,	

Message Tracking

Message Tracking tracks messages sent by Rhapsody to an external system. A tracking scheme can be used on a communication point configured in either Out->In or Bidirectional mode.

A unique message identifier present in each message sent by Rhapsody is used to reconcile it with the matching acknowledgement response. In an HL7 message sent by Rhapsody, the MSH.10 (Message Control Id) is normally used as the message identifier. This is matched to the MSA.2 (Message Control Id) of the acknowledgement message received by Rhapsody and is used to match the messages. A tracking scheme also permits actions to be taken based on status information contained in the response message. For example, an error response could be used to stop further processing or notify an administrator.

Tracking modes

The mode of the communication point (either Out->In or Bidirectional) where message tracking is applied will determine if it functions synchronously or asynchronously.

Synchronous Message Tracking

Synchronous message tracking is used when the communication point is configured in Out->In mode. In this mode, Rhapsody does not send the next message until a response for

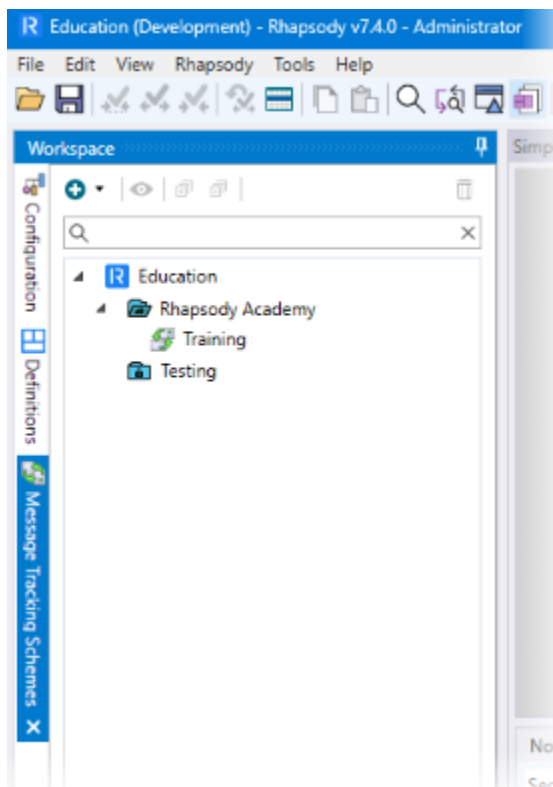
the previous message has been received. If the response received indicates that the original message should be resent, this is done immediately, before any other messages are processed by the communication point. Some systems require a synchronous mode to be used, as they are unable to process new messages while the previous message is still being processed.

Asynchronous Message Tracking

Asynchronous message tracking is used when the communication point is configured in Bidirectional mode. When operating in asynchronous mode, Rhapsody does not wait for a response before sending the next message. This means messages are sent faster from Rhapsody as it does not have to wait for an external system. In this situation, an external system will continue to receive messages before it has completed processing the previous message, and so the received responses to these messages may arrive out of order as well.

Tracking Scheme Manager

Message Tracking schemes are created and managed from within the Rhapsody IDE from the Message Tracking Scheme Workspace tab, accessible from the **Managers** drop-down or the **View** menu.



Creating a New Message Tracking Scheme

Message Tracking relies on a message definition to work. If a definition already exists in Rhapsody for the messages to be tracked and their response messages, then this should be used. Otherwise, a new message definition must be created before proceeding.

Message Tracking schemes are created and managed from the **Message Tracking Schemes** tab in the Workspace. This tab is accessed from the **View** menu or **Managers** drop-down. When you create a new Message Tracking scheme, the Tracking Scheme editor is displayed. The General tab is first displayed when you can provide the tracking scheme with an appropriate name. Selecting the **Messages** tab presents a screen where details of the messages to be tracked can be defined.

The screenshot shows the 'Message Tracking Scheme: Training/Academy PAS' dialog box with the 'Messages' tab selected. The 'Tracked messages definition' is 'Academy-HL7v2.4-ADT.s3d' and the 'Response messages definition' is 'Academy-HL7v2.4-ADT.s3d'. The 'Tracked messages' section lists 'ADTA01, ADTA03, ADTA28, ADTA29, ADTA30, ADTA31' with 'Message Control ID: MSH.MessageControlID' and 'Response messages: <Any response message>'. The 'Response messages' section lists 'ACK, ACKA01, ACKA03, ACKA28, ACKA29, ACKA30, ACKA31' with 'Response Control ID: MSA.MessageControlID' and 'Response Code: MSA.AcknowledgementCode'. The dialog has buttons for 'Rename', 'OK', 'Cancel', 'Apply', and 'Help'.

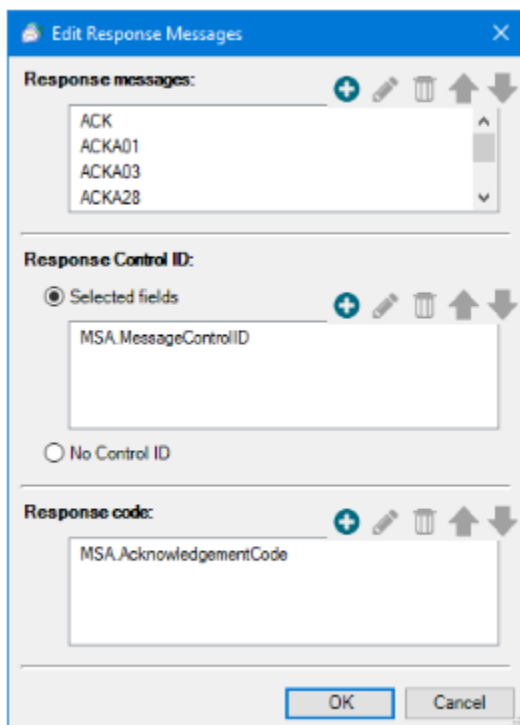
Add Response Messages

Select the plus button to the right of the Response messages section to open the **Edit Response Messages** dialog box. For message formats such as HL7 where all response messages have a common format, all response messages will normally be added to the same response messages set. For a format where this is not the case, a new response message set will be required for each type of response message.

Selecting the **Add** icon for the **Response Messages** lets you define the set of valid response messages.

The **Response Control Id** and **Response Code** are selected by selecting the respective **Add** icons and selecting the appropriate field in the message definition that will

contain this information. These fields will define where Rhapsody looks for the message id and response code in a response message.



Response Messages Editor

Add Tracked Messages

Select the **Add** icon to the right of the Tracked messages section to open the **Edit Tracked Messages** dialog box. Where a message must have a particular response message, it needs to be added individually to the set of tracked messages.



Example showing multiple tracked messages

Remember to specify the message id for each message type created so that Rhapsody is able to match it to the correct response message.

Rhapsody automatically adds the message control id to the outgoing message as an indexed property. This makes it possible to search for the message in the Output Search using the identifier.

Managing Response Messages

Rhapsody can be set up to perform one of a number of actions depending on a response code received in the response message. The actions are defined on the **Response Handling** tab.

An existing action can be edited or deleted by opening the **Response Handling** tab, selecting the action to edit, then choosing one of the buttons on the right of the screen.



A new action is added by selecting the icon.

Message Tracking Scheme: Training/Academy PAS

General Messages Response Handling Error Handling

Response Code Actions

AA

Ack Type: **ACK**
Resend original: No
Action when received: No further action
Notify: No

AE

Ack Type: **ERROR**
Resend original: Yes
Action after resends: Treat as error
Notify: No

AR

Ack Type: **NACK**
Resend original: No
Action when received: No further action
Notify: No

Maximum number of resends: 5
Delay between resends (seconds): 60

☒ Route response messages

Rename OK Cancel Apply Help

Message Tracking Scheme Responses tab

Success Responses

Most response messages received will indicate that the tracked message was successfully processed. In this case, a **Response Code Action** should be created that performs no further action.

The configuration of the Response Code Action for a successful acknowledgement

Error Responses

Error response messages can take many forms depending upon the nature of the external system and how errors are reported. Normally, error responses will require one of the three following behaviors:

Behavior	Usage	Configuration
Resend message	When the external system has reported a temporary fault (for example, the external system cannot connect to its database) and that resending the message may succeed.	Select the Resend original message checkbox. Set Action when received to Shutdown Communication Point . The shutdown action will prevent further processing of messages if Rhapsody continues to receive the same response message.

Behavior	Usage	Configuration
Perform specific processing on error responses	When the external system has reported a fault in the tracked message (such as an invalid date).	Set Action when received to Treat as Error . Error messages will be sent to the communication point's error connector for further processing.
Stop further processing	When the external system has reported a fault that requires manual intervention before it is safe to send further messages, or when the external system responds with an unknown error type.	Set Action when received to Shutdown Communication Point .

When a response action is set to resend the tracked message, the **Maximum number of resends** and **Delay between resends (seconds)** options are used to determine how many times Rhapsody will attempt resend the original message before giving up and performing the action defined by the response code action.

The **Route response messages** options determine if response messages will be sent from the communication point to the route for processing. Most message tracking schemes will leave this option selected, so that response messages can be processed by the Rhapsody route.

Managing Errors

The tracking scheme also needs to be able to deal with the various error conditions that might occur. These are configured from the **Error Handling** tab of the tracking scheme.

Message Tracking Scheme: Training/Academy PAS

General Messages Response Handling Error Handling

No Response

Timeout before resending (sec): 60 Maximum number of resends: 5

Action after maximum resends: Treat as error Browse...

Other

Unrecognized original message: Treat as error Browse...

Duplicate response message: Ignore Browse...

Late response message: Discard message Browse...

Unsolicited response message: Resend original message (works only in Out-In mode) Browse...

Rename OK Cancel Apply Help

Tracking Scheme editor Errors tab

The **No Response** settings determine what Rhapsody should do if a response message is not received within the specified timeframe. After the maximum number of resends the tracking scheme can be configured to perform one of the following options:

The	Setting	Behavior	various
	No further action	Rhapsody processes the next message waiting to be sent to the communication point as if the previous message had been successfully acknowledged.	
	Treat as error	Any message that does not receive a response will be sent to the communication point's error connector. Once the message has been sent to the error connector, further messages will be processed as normal. This option is useful if further processing of unacknowledged messages is required (for example, to notify the intended recipient via an alternative communications channel).	
	Shutdown Communications Point	If a message does not receive a response, the communications point will be stopped. This will prevent further messages from being sent to the external system until an administrator restarts the communication point.	

settings under **Other** determine how Rhapsody should deal with each type of unexpected tracked or response message.

Adding a Tracking Scheme to a Communications Point

The final task once a tracking scheme has been created is to add it to a communication point. A tracking scheme can be added to any communication point that sends messages to an external system (either Out->In or Bidirectional mode).

To add a tracking scheme to a communication point, display the **Message Tracking** tab. Select the **Enable Message Tracking for this communication point** checkbox followed by the **Scheme** from the drop-down list.

The screenshot shows the 'PACS System Properties' dialog box with the 'Message Tracking' tab selected. The dialog has several tabs: General, Connection, Management, Configuration, Scheduling, Message Tracking, Notes, Support Notes, and Auxiliary Files. The 'Message Tracking' tab is active, displaying instructions: 'You can specify a scheme according to which messages will be correlated with responses, configured actions executed based on response type and cases handled, such as timeouts and invalid messages.' Below this, the checkbox 'Enable message tracking for this communication point' is checked. A 'Scheme' section contains a dropdown menu with 'Academy PAS' selected, and buttons for 'View', 'Override', and 'New'. A 'Summary' section lists: 'Definition: Academy-HL7v2.4-ADT.s3d', 'Tracked messages: ADTA01, ADTA03, ADTA28, ADTA29, ADTA30, ADTA31', 'Response messages: ACK, ACKA01, ACKA03, ACKA28, ACKA29, ACKA30, ACKA31', 'Response codes: AA, AE, AR', and 'Response timeout: 60 s'. At the bottom are 'Help', 'OK', and 'Cancel' buttons.

The same tracking scheme can be used against multiple communication points, so long as they all send the same type of message to a compatible external system. A tracking scheme added to multiple communication points will work independently on each one.

When a tracking scheme is added to a communications point, the tracking scheme will override any Out->In properties defined under the **General** tab (although the communication point will still adhere to the appropriate send/receive behavior).

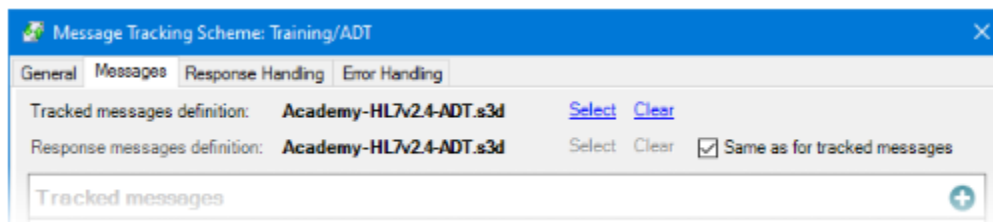
Creating an HL7 Scheme

Message tracking schemes are frequently used to make communication of HL7 messages between systems reliable. As all HL7 messages have a similar message header structure,

all HL7 message tracking schemes will have a similar configuration. Below, we illustrate the normal configuration for a message tracking scheme that tracks HL7 messages.

Message Definitions

Most HL7 message definitions used in Rhapsody will contain definitions for both the messages and their associated acknowledgement messages in the same file. When this is the case, a definition is loaded using the **Select** link associated with the **Tracked message definition** to load a definition. The **Same as for tracked messages** check box is selected to indicate that this definition contains the response message definitions.



If your acknowledgement messages are contained in a separate message definition, the **Same as for tracked message** checkbox is unchecked, and the response definition is loaded using the **Select** link associated with the **Response message definition** field.

Tracked Messages

We will normally create a single set of tracked messages that contains all messages that we could send to an external system. The **Message Control ID** in HL7 is always found in the MSH.MessageControlID field. Where we are not concerned about matching each message with a particular response type, we can select the **Any response message** option to indicate that any response message will be valid for a tracked message.

It is possible to use the **All non-response messages** option to define the set of the tracked message. However, you might find it better to specify all tracked messages as this clearly indicates what messages we expect to send to the external system.

Edit Tracked Messages

Tracked messages:

☒ Selected messages

- ADTA01
- ADTA03
- ADTA28
- ADTA29

☐ All non-response messages

Message Control ID:

☒ Selected fields

- MSH.MessageControlID

☐ No Control ID

Valid responses:

☐ Selected responses

☒ Any response message

OK Cancel

Complex Message Control IDs

In HL7, a message control id is only unique within a particular application. Where is it considered necessary to make the message control id in the tracking scheme unique to a sending system, the MSH.SendingApplication.NamespaceID and MSH.SendingFacility.NamespaceID message fields are added to the **Message Control ID** field. Internally, Rhapsody will concatenate all message fields together to generate the message control id used for tracking.

Message Control ID:

☒ Selected fields

- MSH.SendingApplication.NamespaceID
- MSH.SendingFacility.NamespaceID
- MSH.MessageControlID

☐ No Control ID

Valid responses:

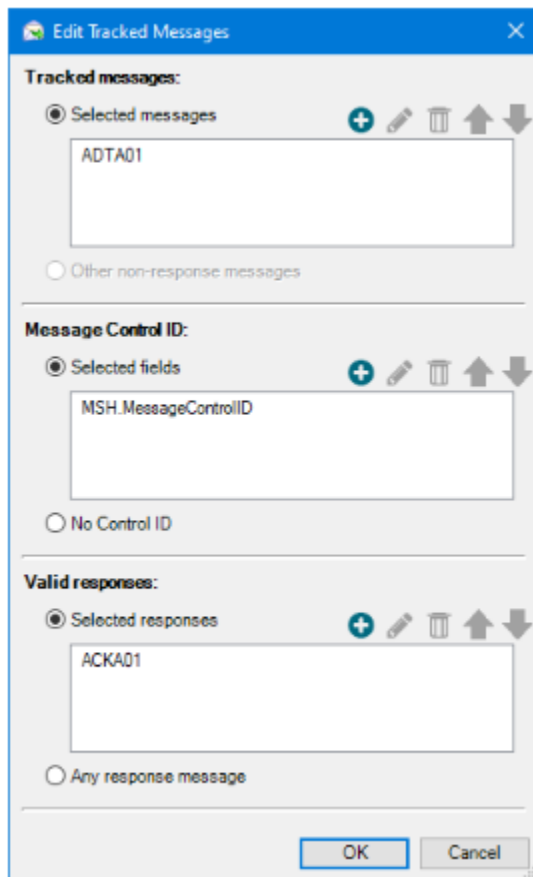
☐ Selected responses

An external system must include these fields in the response message for message tracking to be able to correctly match the tracked message with its response.

Associating a message with a particular response

Where it is important that each response message is of the correct type for a particular tracked message, we can create a multiple tracked message. Each tracked message will contain just one message type and one associated valid response message type.

Where multiple tracked messages are created, the message tracking scheme will display a list containing each tracked message and the associated response messages:



The screenshot shows a Windows-style dialog box titled "Edit Tracked Messages". It contains three main sections, each with a list of items and a set of icons (add, edit, delete, up, down) to the right of the list.

- Tracked messages:** The first section has a radio button selected for "Selected messages". Below it is a list box containing the text "ADTA01".
- Message Control ID:** The second section has a radio button selected for "Selected fields". Below it is a list box containing the text "MSH.MessageControlID".
- Valid responses:** The third section has a radio button selected for "Selected responses". Below it is a list box containing the text "ACKA01".

At the bottom of the dialog box are two buttons: "OK" and "Cancel".

Tracked messages definition: [Academy-HL7v2.4-ADT.a](#) [Select](#) [Clear](#)

Response messages definition: [Academy-HL7v2.4-ADT.a](#) [Select](#) [Clear](#) ☒ Same as for tracked messages

Tracked messages	
ADTA01	Message Control ID: MSH.MessageControlID Response messages: ACKA01
ADTA03	Message Control ID: MSH.MessageControlID Response messages: ACKA03
ADTA28	Message Control ID: MSH.MessageControlID

Response Messages

The response messages will contain the set of all response messages that could be received in response to a tracked message. As all response messages in HL7 have a similar structure, you will normally only need a single set of response messages. The **Response Control ID** will be MSA.MessageControlID (note that this field is in the message acknowledgement segment), while the **Response code** will be MSA.AcknowledgementCode.

Edit Response Messages

Response messages:

- ACK
- ACKA01
- ACKA03
- ACKA28

Response Control ID:

☒ Selected fields

MSA.MessageControlID

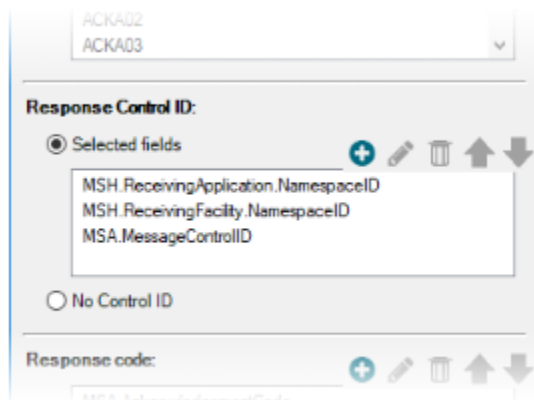
☐ No Control ID

Response code:

MSA.AcknowledgementCode

OK Cancel

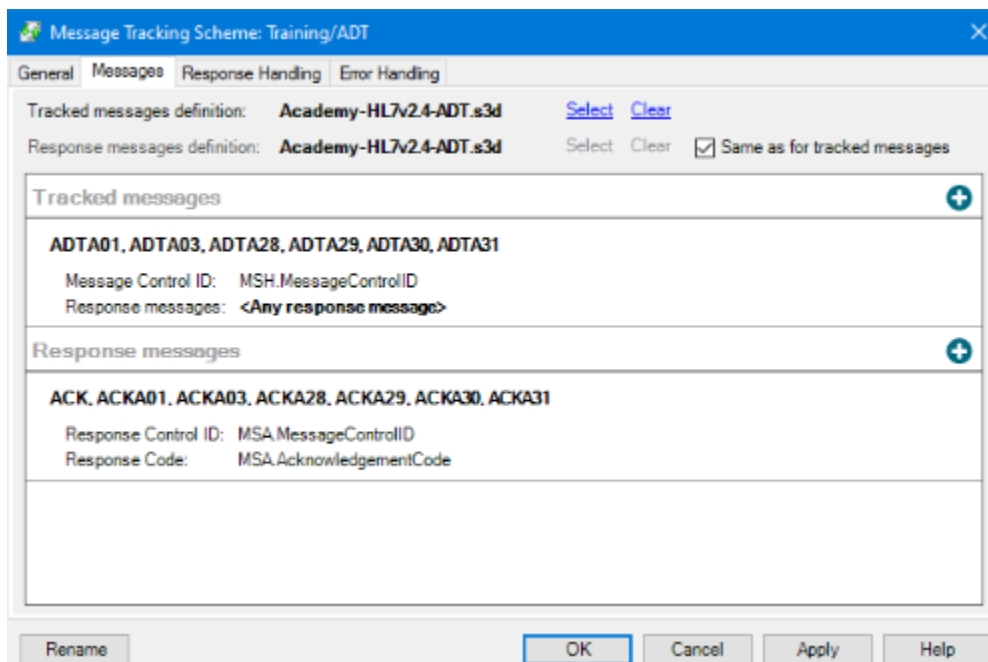
Where a complex Message Control ID was included in the tracked message configuration, matching configuration is needed in the response. The **Response Control ID** needs to match the tracked control id. It is typical for a system to copy the contents of the sending application and sending facility fields to the receiving application and receiving facility fields. In this case, the MSH.ReceivingApplication.NamespaceID and MSH.ReceivingFacility.NamespaceID message fields are added to the **Response Control ID** field.



A screenshot of a configuration window for 'Response Control ID'. At the top, there is a dropdown menu with 'ACKA02' and 'ACKA03' selected. Below this, the 'Response Control ID' section has two radio buttons: 'Selected fields' (which is selected) and 'No Control ID'. Under 'Selected fields', there is a list box containing three items: 'MSH.ReceivingApplication.NamespaceID', 'MSH.ReceivingFacility.NamespaceID', and 'MSA.MessageControlID'. To the right of the list box are icons for adding, editing, deleting, and moving items. Below the radio buttons is a 'Response code:' label and a text field containing 'MSA.AcknowledgementCode'.

The Complete Message Definition

A complete message tracking scheme configuration for an HL7 definition will typically look similar to this:



A screenshot of the 'Message Tracking Scheme: Training/ADT' dialog box. It has four tabs: 'General', 'Messages', 'Response Handling', and 'Error Handling'. The 'Messages' tab is active. It shows 'Tracked messages definition: Academy-HL7v2.4-ADT.s3d' and 'Response messages definition: Academy-HL7v2.4-ADT.s3d'. There are 'Select' and 'Clear' buttons for both, and a checked box for 'Same as for tracked messages'. Below this, there are two main sections: 'Tracked messages' and 'Response messages'. The 'Tracked messages' section lists 'ADTA01, ADTA03, ADTA28, ADTA29, ADTA30, ADTA31' with 'Message Control ID: MSH.MessageControlID' and 'Response messages: <Any response message>'. The 'Response messages' section lists 'ACK, ACKA01, ACKA03, ACKA28, ACKA29, ACKA30, ACKA31' with 'Response Control ID: MSA.MessageControlID' and 'Response Code: MSA.AcknowledgementCode'. At the bottom, there are buttons for 'Rename', 'OK', 'Cancel', 'Apply', and 'Help'.

Response Handling

An HL7 scheme will normally contain response handling rules for AA, AR, & AE response codes. The action taken for each response type will depend on the design of the external system and how Rhapsody needs to be configured for each type of response.

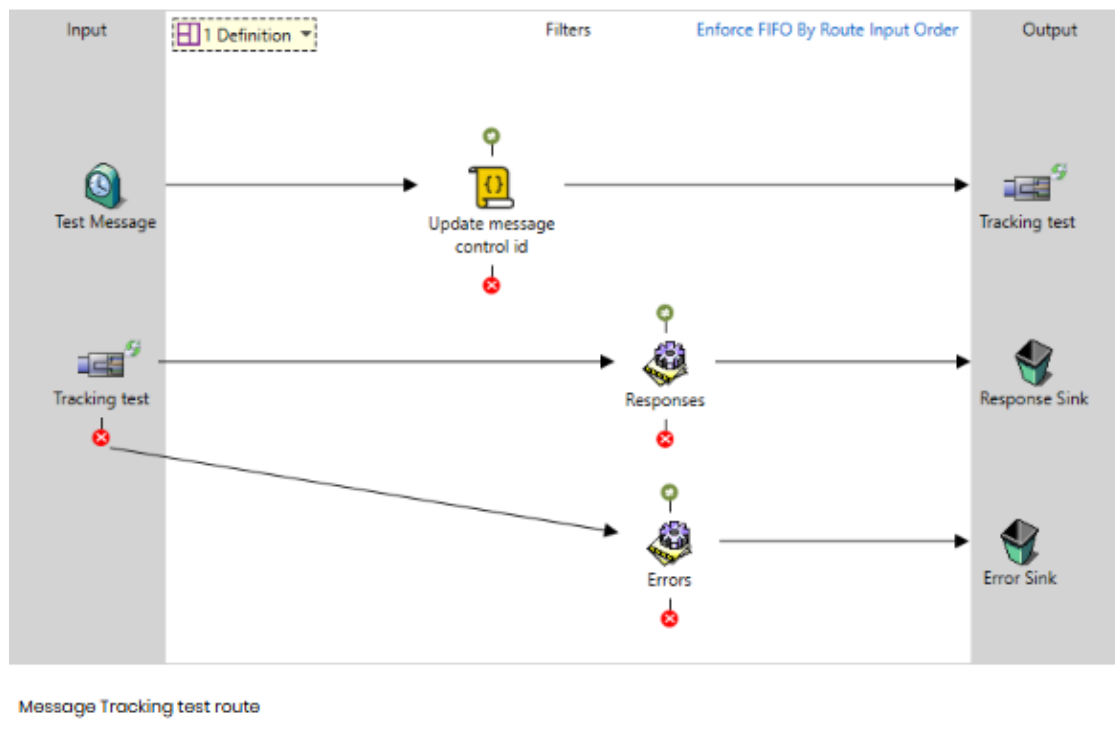
Exercise - Message Tracking

Introduction

In this exercise, we will create a route that sends messages to an external system via the **TCP Client** communication point. We will create a **Message Tracking Scheme** on the outbound communication point to track and manage the message. The EDI Explorer will be used to receive and respond to messages. By changing the way **EDI Explorer** responds to each message, we can investigate the behaviors of the Message Tracking Scheme.

Construct Test Route

We will start the exercise by creating a test route that generates and sends test messages.



Configure the route as follows.

Import the Academy-HL7v2.4-ORU.s3d definition into Rhapsody and associate it with the route.

Timer Communication Point

A Timer communication point is configured to load a test file containing a set of test messages at a rate of one every 30 seconds.

Option	Property
Body Template File	ORUR01Sample.edi
Refresh Rate	30000

JavaScript Filter

The JavaScript filter is configured to insert a unique identifier into the MSH.MessageControlID segment of every message. The JavaScript filter should be configured as follows:

```
// Create the output message
```

```
const next = output.append(input[0]);
```

```
// Generate new message control id user counter and update message
```

```
const ctlID = incCounter("REN209-MsgCtrlId",1);
```

```
next.setField("MSH/MessageControlID", ctlID);
```

This will insert a unique identifier into the MSH.MessageControlID segment of every message. By doing this, we ensure that the message tracking scheme is able to track each message we send from our route.

TCP Client Communication Point

Create a new **TCP Client** communication point. The communication point must be configured in Out->In mode.

Option	Property
Remote Host	localhost

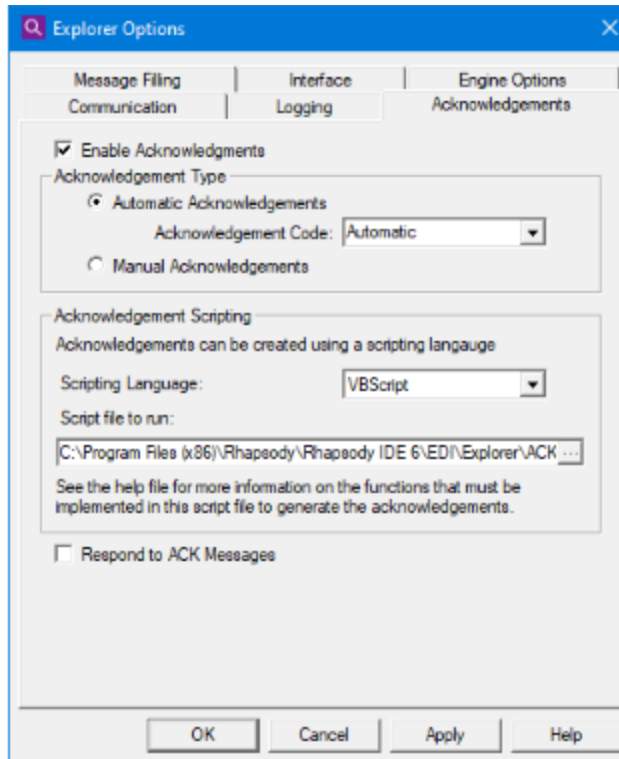
Option	Property
Remote Port	<i>Any value of your choosing that is not already in use.</i>

Testing the Route

Configure EDI Explorer

We will use EDI Explorer to receive and respond to all messages in this exercise. This will let us control how response messages are sent to Rhapsody. This will help us understand the behavior of message tracking.

- Import the Academy-HL7v2.4-ORU.s3d message definition. This is the same definition used in the route. It is possible to open this definition from within the Rhapsody IDE by opening the Definitions tab, right-clicking on the definition and selecting Open With EDI Explorer.
- Configure EDI Explorer to automatically generate acknowledgements with an AA code. This is performed by navigating to **Tools > Options > Acknowledgements** and selecting **Automatic Acknowledgements**.
- Set up **EDI Explorer** to act as a TCP Server and set the listening port number to match the port number that was set when you configured the TCP Client communication point. Once set up, connect to start listening for incoming connections.



Checking the Route

Make sure all changes have been checked in. Start the route and all communication points.

With the **EDI Explorer** instance running, start the route. You should see that:

- Messages are regularly sent to the **EDI Explorer** at the same rate that messages are created by the Timer communication point. Each message contains a unique message control id.
- An acknowledgement is generated and returned by the **EDI Explorer** to the TCP Client communication point.

Open the **Management Console** and look at the received response messages:

- The acknowledgement will contain an **Acknowledgement Code AA**.
- The value of the MSH.MessageControlID field in the original message matches the value of the MSA.MessageControlId field in the generated acknowledgement.

Message Body ▾

Tree View ▾

```
MSH
MSA
  ~AcknowledgementCode "AA"
  ~MessageControlID "26"
```

Flat Wire ▾

Show Whitespace ▾ Encoding: UTF-8

Size: 75 bytes

```
MSH|^~\&|||^City Hospital^HL7|20180813155303||ACK^R01|9948||2.4\r
MSA|AA|26\r
```

Configuring Message Tracking

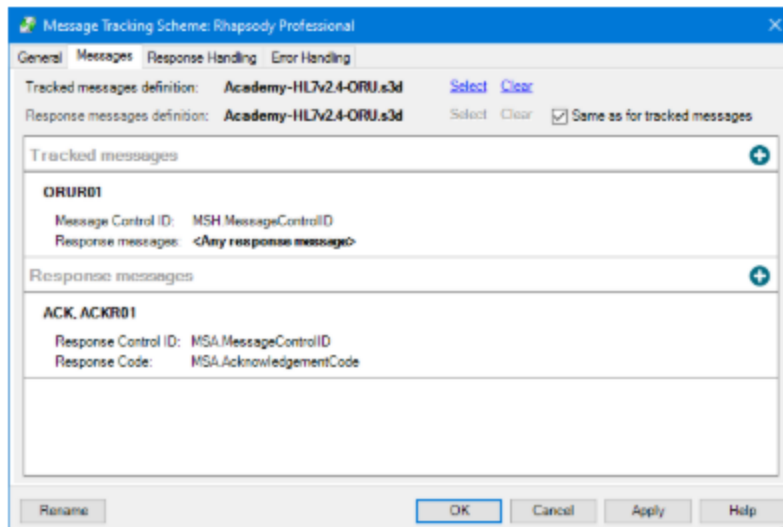
Open the **Message Tracking Scheme** Manager from the **Managers** drop-down or the **View** menu and select the **Add** button to create a new message tracking scheme. Provide a unique and informative name for the new scheme.

Messages

We will start by defining the messages that our message tracking scheme will manage.

Messages Definition

Select the Academy-HL7v2.4-ORU.s3d message definition as the **Tracked Message Definition**. This is the same definition we used on the route. Ensure the **Same as for tracked messages** checkbox is selected. This tells Rhapsody that the definitions for the incoming messages and outgoing acknowledgements are in the same definition.



Tracked Messages

We need to define the set of messages that will be sent to the external system. For this tracking scheme, we will be sending just one message type – an ORU^R01 message – to the external system.

Select **Add** this icon to edit the **Tracked Messages** and complete using the following details:

Option	Property
Tracked messages	ORUR01
Message Control ID	MSH.MessageControlID
Response Messages	Any response message

Response Messages

We now want to define the set of response messages we expect to receive back from the external system. In this exercise, it could be either an ACK or an ACK^R01 type acknowledgement message. Add these **Response Messages** using the following details:

Option	Property
Ack Type	ACK & ACKR01
Response Control ID	MSA.MessageControlID
Response Code	MSA.AcknowledgementCode

Response Handling

We now need to configure the tracking scheme to understand the three HL7 response codes that will be generated during testing of the exercise. Switch to the **Response Handling** tab of the Message Tracking Scheme's configuration dialog to specify the scheme's behavior when a response with a given code is returned. Enter the following **Response Code Actions**:

AA

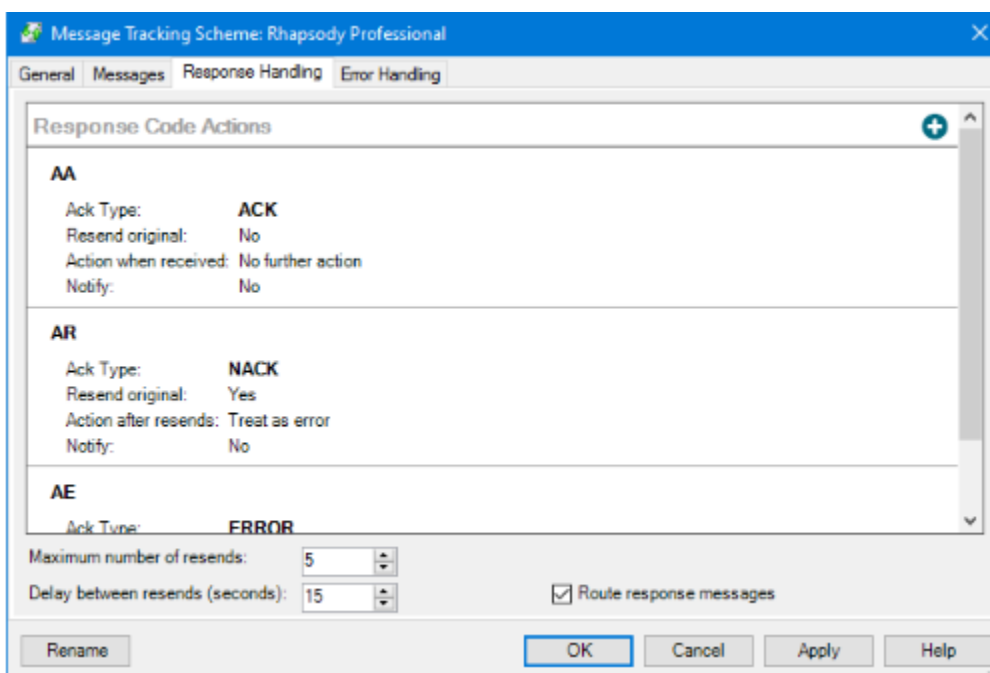
Option	Property
Ack Type	ACK
Resend original	No
Action when received	No further action
Notify	No

AR

Option	Property
Ack Type	NACK
Resend original	Yes
Action after resends	Treat as error
Notify	No

AE

Option	Property
Ack Type	ERROR
Resend original	Yes
Action after resends	Treat as error
Notify	No



Resend options

Next, we need to configure the behavior of the tracking scheme for the acknowledgement types where the Resend original option has been set to yes. In this exercise, that is the AR and AE acknowledgement types.

Option	Property
Maximum Number of resends	5

Option	Property
Delay between resends (seconds)	15
Route Response Messages	☒ (Checked)

Error Handling

We now need to switch to the **Error Handling** tab. On this tab, we will define how we want the message tracking scheme to behave when an error is detected.

No Response

The no response settings let us define the behavior when a tracked message is sent, but no response is received. We want Rhapsody to attempt to resend a message twice at 30 second intervals. After the two resend attempts, we will send the message to the communication point's error connector.

Other

At this stage in the exercise, we won't be configuring any of the other error settings, so we can leave them all with a behavior of Ignore.

Option	Property
Timeout before resending (sec)	30
Maximum number of resends	2
Action after maximum resends	Treat as error

Check in Changes

We have now finished configuring the message tracking scheme. Close the configuration dialog by selecting **OK**. Select **Check In** to check in your new message tracking scheme.

Add Message Tracking Scheme to Communication Point

Return to the route created at the beginning of this exercise. A Message Tracking Scheme is always associated with a communication point that sends messages to an external system. In the exercise, this is the Tracking Test **TCP Client** communication point. Open the **Properties** dialog for the communication point, then switch to the **Message Tracking** tab. Select the **Enable Message Tracking for this Communication Point** checkbox.

From the **Scheme** drop-down, select the tracking scheme created on the previous page. Note the summary of the scheme that is displayed and that there is a **View** button which you can select to see the full details of the scheme.

The screenshot shows the 'Tracking test Properties' dialog box with the 'Message Tracking' tab selected. The dialog has several tabs: General, Connection, Management, Configuration, Scheduling, Message Tracking (active), Notes, Support Notes, and Auxiliary Files. Below the tabs, there is a text box explaining that a scheme can be specified for correlating messages with responses. A checkbox labeled 'Enable message tracking for this communication point' is checked. Below this, there is a 'Scheme' section with a dropdown menu showing 'Rhapsody Professional'. To the right of the dropdown are 'View', 'Override', and 'New' buttons. Below the 'Scheme' section is a 'Summary' section containing the following information: Definition: Academy-HL7v2.4-ORU.s3d, Tracked messages: ORUR01, Response messages: ACK, ACKR01, Response codes: AA, AE, AR, and Response timeout: 60 s. At the bottom of the dialog are 'Help', 'OK', and 'Cancel' buttons.

Tracking test Properties

General Connection Management Configuration Scheduling **Message Tracking** Notes Support Notes Auxiliary Files

You can specify a scheme according to which messages will be correlated with responses, configured actions executed based on response type and cases handled, such as timeouts and invalid messages.

☒ Enable message tracking for this communication point

Scheme

Scheme: Rhapsody Professional

View Override New

Summary

Definition: Academy-HL7v2.4-ORU.s3d

Tracked messages: ORUR01

Response messages: ACK, ACKR01

Response codes: AA, AE, AR

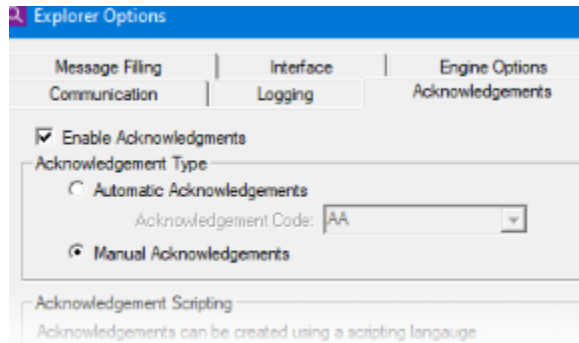
Response timeout: 60 s

Help OK Cancel

Testing Message Tracking

Configure EDI Explorer

We need to modify the response behavior of EDI Explorer so that we can control how and when response messages are generated and returned. Open EDI Explorer and navigate to Tools > Options > Acknowledgements. Change the Acknowledgement Type setting to Manual Acknowledgements. Save the configuration changes by selecting OK.



Response Tracking Test

We will test our message tracking scheme with three different tests.

AA Response

In our first test of the message tracking scheme, we will respond to each message in EDI Explorer with an AA response code. We will then inspect the messages in the Management Console to see how tracked messages are presented.

- Make sure all Rhapsody configuration for this exercise is started. Wait for one message to be generated by the **Timer** communication point in your configuration, then stop just that component. This makes it easier to see what is going on while testing the message tracking scheme.

From within **EDI Explorer**, wait for a message to arrive. On the incoming message, make sure AA is selected from the drop-down in the top right-hand corner of the message window, select **Send Immediately**, then select the send icon to generate and send an acknowledgement to the Rhapsody configuration.

- Open the Rhapsody **Management Console** and view the output archive for the TCP Client communication point. Select the first message in the list. This will display the message you just acknowledged in the EDI Explorer.

Notice how the message event for the TCP Client communication point lists the message control id for the message and states that it was positively acknowledged.

AE Response

For our second test of the tracking scheme, we will repeatedly respond to the messages with an AE code. This will cause Rhapsody to resend the test messages a number of times.

Send a test message into the route and select an acknowledgement of type AE to be returned to the test configuration. What you should see is:

- Start the **Timer** communication point until one message has been generated, then stop it again.
- From within **EDI Explorer**, change the acknowledgement type to AE, then generate and send a reply to the test configuration. After 15 seconds, the original message will be resent to EDI Explorer again (we could verify this by looking at the MSH.MessageControlID). Generate and send an acknowledgement with the same AE code. Repeat this process another four times to send an AE response for each resend of the original message.
- Open the Rhapsody Management Console and view the output archive for the **TCP Client** communication point. View the most recent message in the list.

COMPONENT	TIMESTAMP	DESCRIPTION
Test Message	Jun 15, 2020 16:17:10.588	Received by communication point
Test Route	Jun 15, 2020 16:17:10.591	Placed on route
	Jun 15, 2020 16:17:10.603	Property extraction performed
Update message control id	Jun 15, 2020 16:17:10.621	Filtering succeeded; took 17.846 ms
Tracking test	Jun 15, 2020 16:17:10.648	Delivered to communication point
	Jun 15, 2020 16:17:10.648	Registered for message tracking (Type=ORUR01, Control id=45)
	Jun 15, 2020 16:17:10.665	Sent by communication point, total processing took 0.077 seconds.
	Jun 15, 2020 16:17:21.902	Error Response (Type=ACKR01, Response Code=AE)
	Jun 15, 2020 16:17:36.920	Sent by communication point, total processing took 26.332 seconds.
	Jun 15, 2020 16:17:45.086	Error Response (Type=ACKR01, Response Code=AE)
	Jun 15, 2020 16:18:00.096	Sent by communication point, total processing took 49.508 seconds.
	Jun 15, 2020 16:18:06.589	Error Response (Type=ACKR01, Response Code=AE)
	Jun 15, 2020 16:18:21.629	Sent by communication point, total processing took 71.041 seconds.
	Jun 15, 2020 16:18:24.405	Error Response (Type=ACKR01, Response Code=AE)
	Jun 15, 2020 16:18:39.409	Sent by communication point, total processing took 88.821 seconds.
	Jun 15, 2020 16:18:41.967	Error Response (Type=ACKR01, Response Code=AE)
	Jun 15, 2020 16:18:56.969	Sent by communication point, total processing took 106.381 seconds.
	Jun 15, 2020 16:19:00.793	Error Response (Type=ACKR01, Response Code=AE)
Tracking test	Jun 15, 2020 16:19:00.797	Delivered to the failed queue of the communication point
Test Route	Jun 15, 2020 16:19:00.799	Placed on route
	Jun 15, 2020 16:19:00.800	Property extraction performed
Errors	Jun 15, 2020 16:19:00.800	Filtering succeeded; took 0.009 ms
Error Sink	Jun 15, 2020 16:19:00.800	Delivered to communication point
	Jun 15, 2020 16:19:00.802	Sent by communication point, total processing took 0.006 seconds.

Management Console showing a message with an AE response

Notice that the message events show the message being sent to EDI Explorer 6 times. After the last send failure, the message is redirected to the communication point's error connector, which sends it to the error sink. Have a look at the message properties to see how the error is reported.

Error Tracking Test

In the final test of the message tracking scheme, we will see what happens when a message is not acknowledged.

- Start the **Timer** communication point until one message has been generated (it is not necessary to stop the communication point for this test).
- From within **EDI Explorer** watch the test message arrive. Do not respond to the message. After 30 seconds, a second copy of the message will arrive in EDI Explorer.
- Open the Rhapsody **Management Console** and view the output archive for the **TCP Client** communication point. View the most recent message in the list.

EVENT	TIMESTAMP	DESCRIPTION
Test Message	Jun 15, 2020 16:55:20.887	Received by communication point
Test Route	Jun 15, 2020 16:56:20.891	Placed on route
	Jun 15, 2020 16:56:20.900	Property extraction performed
Update message control id	Jun 15, 2020 16:56:20.925	Filtering succeeded; took 24.876 ms
Tracking test	Jun 15, 2020 16:56:20.933	Delivered to communication point
	Jun 15, 2020 16:56:20.933	Registered for message tracking (Type=ORUR01, Control id=46)
	Jun 15, 2020 16:56:20.951	Sent by communication point, total processing took 0.064 seconds.
	Jun 15, 2020 16:56:50.999	Timeout while waiting for response
	Jun 15, 2020 16:56:51.000	Sent by communication point, total processing took 30.113 seconds.
	Jun 15, 2020 16:57:21.002	Timeout while waiting for response
	Jun 15, 2020 16:57:21.003	Sent by communication point, total processing took 60.116 seconds.
	Jun 15, 2020 16:57:51.016	Timeout while waiting for response
	Jun 15, 2020 16:57:51.017	Tracking failure - no response received
Tracking test	Jun 15, 2020 16:57:51.017	Delivered to the failed queue of the communication point
Test Route	Jun 15, 2020 16:57:51.018	Placed on route
	Jun 15, 2020 16:57:51.018	Property extraction performed
Errors	Jun 15, 2020 16:57:51.018	Filtering succeeded; took 0.009 ms
Error Sink	Jun 15, 2020 16:57:51.019	Delivered to communication point
	Jun 15, 2020 16:57:51.019	Sent by communication point, total processing took 0.003 seconds.

Management Console showing a message without a response

Notice that the message events show the message being sent to EDI Explorer 3 times at 30 second intervals. After the last send failure, the message is redirected to the communication point's error connector, which sends it to the error sink. Have a look at the message properties to see how the error is reported.